UNIVERSITE DE YAOUNDE I
****
ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE
****
DEPARTEMENT DE GENIE INFORMATIQUE

UNIVERSITY OF YAOUNDE I
****
NATIONAL ADVANCED SCHOOL OF ENGINEERING
****
DEPARTMENT OF COMPUTER ENGINEERING

CONCEPTION ET DÉPLOIEMENT D'UNE SOLUTION DE SUPERVISION RÉSEAU BASÉE SUR ZABBIX : CAS DE LA CAISSE DES DÉPÔTS ET CONSIGNATIONS DU CAMEROUN

Mémoire de fin d'études

Présenté et soutenu par :

MINKANDE BELINGA DANIEL ERIC
21P601

En vue de l'obtention du :

Diplôme d'Ingénieur de Conception en Management des Systemes d'Information

Sous la supervision de :

DR KAMENI HOMTE JAURES, CHARGÉ DE COURS, ENSPY, UY1

Devant le jury composé de :

Président : **M. PROFESSEUR, UY1**

Rapporteur : **DR KAMENI HOMTE JAURES, CHARGÉ DE COURS, ENSPY, UY1**

Examinateur : **M. CHARGÉ DE COURS, UY1**

Invité : **NANGUH FRANKLIN, INGENIEUR, CDEC**

Année académique :2026-2027

CONCEPTION ET DÉPLOIEMENT D'UNE SOLUTION DE SUPERVISION RÉSEAU BASÉE SUR ZABBIX : CAS DE LA CAISSE DES DÉPÔTS ET CONSIGNATIONS DU CAMEROUN

Mémoire de fin d'études

Présenté et soutenu par :

MINKANDE BELINGA DANIEL ERIC
21P601

En vue de l'obtention du :

Diplôme d'Ingénieur de Conception en Management des Systemes d'Information

Année académique :2025-2026

Mémoire soutenu le :18 Août 2026

DÉDICACE



REMERCIEMENTS

Ce travail n'aurait pas été possible sans l'aide de nombreuses personnes à qui je tiens à exprimer ma profonde gratitude :

- **[Titre, Nom du Président du Jury]**, qui me fait l'insigne honneur de présider ce jury ;
- **[Titre, Nom des Examinateurs]**, pour avoir accepté d'examiner ce travail et pour l'intérêt porté à ce projet ;
- **Dr, KAKMENI HOMTE Jaures**, pour son suivi constant, ses conseils avisés et l'expertise qu'il a partagée tout au long de ce projet ;
- **M. RICHARD EVINA OBAM**, Directeur Général de la CDEC, pour m'avoir permis d'effectuer mon stage de fin d'études au sein de la CDEC ;
- **M. FRANKO ATEKWANA**, Directeur des Systèmes d'Information, pour m'avoir accueilli au sein de sa division et m'avoir offert les conditions nécessaires à la réalisation de ce travail ;
- **M. NANGUH FRANKLIN**, pour son assistance technique, sa disponibilité et ses recommandations précieuses durant mon stage ;
- Tout le corps enseignant de l'**École Nationale Supérieure Polytechnique de Yaoundé**, notamment celui du Département de **Génie Informatique**, pour la qualité de la formation reçue durant mon cursus d'ingénieur ;
- Mes très chers parents, **M. BELINGA MINKANDE, Mme Pr JACQUELINE ZE MINKANDE, Mme. ASSOMO MINKANDE JUDITH et Mme BIEME ABESSOLO JEANNE, Mme NGA AKONO ASTRID**, pour leur amour inconditionnel, leurs prières et leur soutien multiforme tout au long de mes études ;
- Mes frères et sœurs, **MINKOULOU Etienne, NTYAM COLLINS,JEAN PAUL NKOLO NGONO, EBO'O MARIE DESIRE, EUGENIE FELICITE ABENG Epse EBA'A EFANDENE, PR JUNETTE ARLETTE METOGO MBENGONO,Dr JUNIE ANNICK METOGO NTSAMA , PASCALINE MEDOU, NGA MARIE NOELLE**, pour leurs encouragements , conseils et leur présence constante à mes côtés ;
- Mes mentors **M. EVINA EYA PIERRE PAUL, JEAN ENONDJI** , pour l'impact positif et les conseils qui ont guidé mes pas au quotidien ;
- Ma marrain académique **NGUEFFO MEGANE** et mes aînés, pour leurs retours d'expérience et l'accompagnement durant ma formation ;
- Mes amis et camarades de promotion, **TOLOKOUM DAVID, KAKOH KOUT VIANEL, MBOUI GAMALEU GLORIEUX MICHEL,ANDRE GUSTAVE BITOTE, EDJE'E ONANA JUVENIL**, pour leur soutien, les échanges constructifs et les moments de détente partagés ;
- Enfin, un grand merci à tous ceux qui, de près ou de loin, m'ont apporté leur soutien moral, spirituel ou intellectuel tout au long de cette démarche .

RÉSUMÉ

Les établissements publics financiers camerounais en phase de structuration, à l'image de la Caisse des Dépôts et Consignations du Cameroun (CDEC), opérationnalisée en janvier 2023 après quinze années d'inactivité institutionnelle, développent des infrastructures informatiques croissantes sans disposer d'un dispositif de supervision centralisé, ce qui limite la détection des incidents à un signalement réactif par les utilisateurs et fragilise la continuité de service. Ce mémoire, réalisé dans le cadre d'un stage de fin d'études d'ingénieur de conception au sein de la Direction des Systèmes d'Information de la CDEC, propose la conception, la validation et le déploiement en environnement de simulation d'une solution de supervision réseau et systèmes fondée sur l'outil open source Zabbix 7.0 LTS, combinant supervision par agent logiciel pour les postes et serveurs et supervision par le protocole SNMP pour les équipements réseau actifs (routeur VyOS, pare-feu pfSense), l'ensemble étant reproduit fidèlement dans un laboratoire virtualisé GNS3/VMware Workstation avant toute transposition vers l'infrastructure réelle. Au-delà de la configuration de l'architecture et du mécanisme d'alerte par courriel, ce travail se distingue par une démarche de validation expérimentale : trois scénarios de comportements anormaux contrôlés – force brute d'authentification SSH, scan de reconnaissance réseau et rafale de trafic –, générés depuis un poste de test dédié sous Kali Linux et positionnés par rapport au référentiel international MITRE ATT&CK, ont permis de vérifier de bout en bout le fonctionnement de la chaîne de détection et de notification. Les résultats obtenus confirment un délai de détection et de notification systématiquement inférieur à deux minutes pour les trois scénarios, ainsi que la production d'un guide d'exploitation et de recommandations de sécurisation (migration vers SNMPv3, réétalonnage des seuils) destinés à accompagner la Direction des Systèmes d'Information dans le déploiement autonome de la solution sur son infrastructure de production.

Mots clés : supervision réseau, Zabbix, SNMP, CDEC, MITRE ATT&CK

Abstract

Cameroonian public financial institutions undergoing structural development, such as the Caisse des Dépôts et Consignations du Cameroun (CDEC), operationalized in January 2023 after fifteen years of institutional inactivity, are building growing IT infrastructures without a centralized monitoring system, which limits incident detection to reactive user reporting and undermines service continuity. This dissertation, carried out as part of an engineering end-of-studies internship within the Information Systems Division of CDEC, proposes the design, validation and deployment, in a simulated environment, of a network and systems monitoring solution based on the open-source tool Zabbix 7.0 LTS, combining software-agent monitoring for workstations and servers with SNMP-based monitoring for active network equipment (VyOS router, pfSense firewall), the whole being faithfully reproduced in a virtualized GNS3/VMware Workstation laboratory prior to any transposition to the real infrastructure. Beyond configuring the architecture and the email alerting mechanism, this work stands out through an experimental validation approach : three controlled anomalous-behavior scenarios – SSH authentication brute forcing, network reconnaissance scanning and traffic flooding – generated from a dedicated Kali Linux test host and mapped against the international MITRE ATT&CK framework, were used to verify end-to-end the functioning of the detection and notification chain. The results confirm a detection and notification delay consistently under two minutes across all three scenarios, together with an operations guide and hardening recommendations (migration to SNMPv3, threshold recalibration) intended to support the Information Systems Division in independently deploying the solution on its production infrastructure.

Keywords : network monitoring, Zabbix, SNMP, CDEC, MITRE ATT&CK

Sigles et Abréviations

API	Application Programming Interface (interface de programmation)
CDEC	Caisse des Dépôts et Consignations du Cameroun
CPU	Central Processing Unit (unité centrale de traitement)
DHCP	Dynamic Host Configuration Protocol
DSI	Direction des Systèmes d'Information
ENSPY	École Nationale Supérieure Polytechnique de Yaoundé
GNS3	Graphical Network Simulator 3
IDS	Intrusion Detection System (système de détection d'intrusion)
IETF	Internet Engineering Task Force
IP	Internet Protocol
LAN	Local Area Network (réseau local)
LTS	Long Term Support
MIB	Management Information Base
MSI	Mathématiques et Sciences de l'Ingénieur
MTBF	Mean Time Between Failures (temps moyen entre pannes)
MTTR	Mean Time To Repair (temps moyen de réparation)
NAT	Network Address Translation
NIST	National Institute of Standards and Technology
OID	Object Identifier (identifiant d'objet)
RAM	Random Access Memory (mémoire vive)
RFC	Request For Comments
SMTP	Simple Mail Transfer Protocol
SNMP	Simple Network Management Protocol

SSH	Secure Shell
STARTTLS	Start Transport Layer Security
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
UML	Unified Modeling Language
USM	User-based Security Model
VM	Virtual Machine (machine virtuelle)
VMnet	Virtual Machine Network (réseau virtuel VMware)
WAN	Wide Area Network (réseau étendu)

GLOSSAIRE

Agent (Zabbix Agent2) : logiciel installé localement sur un hôte supervisé, chargé de collecter les métriques systèmes (CPU, mémoire, disque, réseau, journaux) et de les transmettre au serveur Zabbix, en mode actif ou passif. Utilisé dans ce mémoire pour la supervision des postes et serveurs.

Cloud node : nœud spécial de GNS3 permettant de relier une topologie réseau simulée à une interface réseau existante de la machine hôte, notamment un réseau virtuel VMware (*VMnet*). C'est le mécanisme de pontage central de l'architecture de laboratoire de ce mémoire.

Communauté SNMP : chaîne de caractères jouant le rôle de mot de passe faible dans les versions SNMPv1 et SNMPv2c, transmise en clair et permettant l'accès en lecture (*public*) ou en écriture aux variables d'un équipement.

Déclencheur (*trigger*) : expression logique évaluée en continu par le serveur Zabbix sur une ou plusieurs métriques collectées, qui passe à l'état *Problem* lorsqu'une condition anormale est détectée. C'est le mécanisme central de détection d'incident exploité dans ce mémoire.

Disponibilité : proportion du temps pendant laquelle un système ou un équipement est opérationnel, calculée à partir du MTBF et du MTTR (cf. chapitre 1, équation 1.4). Indicateur de référence pour évaluer l'apport de la solution proposée.

Force brute : technique consistant à tester successivement un grand nombre de combinaisons d'identifiants et de mots de passe afin de deviner des informations d'authentification valides. Simulée dans ce mémoire avec l'outil Hydra contre le service SSH d'un hôte supervisé (chapitre 3).

Groupe d'hôtes : regroupement logique d'hôtes au sein de Zabbix, utilisé dans ce mémoire pour distinguer les postes, les serveurs et les équipements réseau de la CDEC, et appliquer des règles d'alerte différenciées selon la criticité de chaque catégorie.

Hyperviseur : logiciel permettant d'exécuter plusieurs systèmes d'exploitation indépendants (machines virtuelles) sur une même machine physique. VMware Workstation joue ce rôle dans le laboratoire de ce mémoire.

Item : dans Zabbix, unité élémentaire de collecte associée à un hôte, définissant précisément quelle donnée est récupérée (par exemple `net.if.in` pour le débit entrant d'une interface réseau) et selon quelle méthode (agent, SNMP, journal).

MIB (*Management Information Base*) : base de données structurée et normalisée décrivant l'ensemble des variables interrogeables par SNMP sur un équipement réseau, organisées de façon hiérarchique sous forme d'OID.

Modèle (*template*) : dans Zabbix, ensemble préconfiguré d'items, de déclencheurs et de graphiques, applicable en une seule opération à un hôte partageant les mêmes caractéristiques (par exemple *Linux by Zabbix agent*).

OID (*Object Identifier*) : identifiant hiérarchique unique, exprimé sous forme d'une suite numérique séparée par des points, désignant une variable précise au sein d'une MIB SNMP (par exemple `1.3.6.1.2.1.1.3.0` pour le temps de fonctionnement d'un équipement).

Polling : méthode de supervision active consistant, pour le serveur de supervision, à interroger périodiquement les équipements surveillés afin d'en récupérer l'état, par opposition à la remontée spontanée d'information (trap).

Rafale réseau (*flood*) : envoi massif et rapide de paquets réseau vers une cible, destiné à saturer sa bande passante ou ses ressources de traitement. Simulée dans ce mémoire avec l'outil hping3 (chapitre 3).

Scan de ports : technique de reconnaissance réseau consistant à sonder systématiquement les ports d'un hôte afin d'identifier les services actifs. Simulée dans ce mémoire avec l'outil Nmap contre un poste supervisé (chapitre 3).

Trap SNMP : notification envoyée spontanément par un équipement vers le serveur de supervision lors d'un changement d'état, constituant le mécanisme de supervision passive du protocole SNMP.

VMnet : réseau virtuel créé et géré par VMware Workstation, utilisé dans ce mémoire en mode *Host-only* pour relier de façon isolée les machines virtuelles du laboratoire à la topologie GNS3, sans passerelle vers le réseau physique de la machine hôte.

Glossaire limité aux termes dont l'usage spécifique dans ce mémoire justifie une clarification ; les sigles simples sont développés dans la liste des abréviations précédente.

TABLE DES MATIÈRES

Dédicace i
Remerciements ii
Résumé iv
Abstract v
Sigles et Abréviations vi
Glossaire vii
Liste des figures x
Liste des tableaux xii

Introduction Générale 1

1 Concepts généraux et État de l'art 5
1.1 Généralités sur la supervision réseau 6
1.1.1 Définitions, finalités et approches de collecte 6
1.1.2 Architecture générique d'un système de supervision 6
1.1.3 L'environnement hétérogène visé : le cas de la CDEC 7
1.2 Protocoles et indicateurs de supervision 8
1.2.1 Le protocole SNMP : architecture, MIB et versions 8
1.2.2 Supervision par agent logiciel et complémentarité avec SNMP 9
1.2.3 Indicateurs de fiabilité : MTBF, MTTR et disponibilité 10
1.3 Virtualisation, simulation et outils de test 10
1.3.1 Virtualisation et simulation de topologies 10
1.3.2 Équipements réseau simulés : VyOS et pfSense 11
1.3.3 Kali Linux et cadre de référence MITRE ATT&CK 11
1.4 Panorama et positionnement des solutions de supervision 12
1.4.1 Panorama comparatif des solutions existantes 12
1.4.2 Bilan et positionnement du projet 13

2 Méthodologie 15
2.1 Démarche méthodologique et analyse de l'existant 16
2.1.1 Principe général et étapes de la démarche 16
2.1.2 Analyse de l'existant et limites de la supervision actuelle 16
2.2 Spécification des exigences du système 17
2.2.1 Exigences fonctionnelles et non fonctionnelles 17

2.2.2 Contraintes techniques et expérimentales 19
2.3 Architecture globale de la solution proposée 19
2.3.1 Environnement hybride et intégration du poste de test Kali Linux 19
2.3.2 Topologie, répartition des rôles et justification des choix technologiques . . 20
2.4 Plan d'adressage et conception de la supervision par équipement 22
2.4.1 Plan d'adressage et segmentation réseau 22
2.4.2 Répartition des méthodes de supervision et organisation des hôtes 23
2.4.3 Mécanisme d'alerte et scénarios de test de sécurité 24
2.5 Modélisation UML de la solution 25
2.5.1 Acteurs et diagramme de cas d'utilisation 25
2.5.2 Diagramme de séquence : cycle de détection et de notification 26
2.6 Démarche de transposition vers la production 27
Bilan du chapitre 28

3 Implémentation et résultats **29**
3.1 Mise en place de l'environnement de virtualisation 30
3.2 Configuration des équipements réseau 32
3.3 Installation et configuration du serveur Zabbix 33
3.4 Déploiement de la supervision des hôtes 34
3.5 Configuration du mécanisme d'alerte 37
3.6 Exécution des scénarios de sécurité 38
3.6.1 Scénario 1 : force brute d'authentification SSH (Hydra) 39
3.6.2 Scénario 2 : scan de reconnaissance réseau (Nmap) 40
3.6.3 Scénario 3 : rafale de trafic réseau (hping3) 41
3.6.4 Synthèse des résultats et mesure du délai de détection 41
3.7 Analyse critique des résultats et recommandations 43
3.8 Démarche de transposition vers la production 44
Bilan du chapitre 44

Conclusion Générale **46**

Références **47**

TABLE DES FIGURES

FIGURE **Page**

1.1 Architecture générique d'un système de supervision réseau 7

2.1 Démarche méthodologique adoptée pour la conception de la solution 16
2.2 Topologie logique cible de l'architecture de supervision. 20
2.3 Diagramme de cas d'utilisation du système de supervision 26
2.4 Diagramme de séquence du cycle de détection d'une alerte 27

3.1 Topologie de supervision de la CDEC opérationnelle dans GNS3 30
3.2 Écran d'accueil de l'assistant d'installation de Zabbix 7.0 34
3.3 Assistant d'installation de l'agent Zabbix sur Windows Server. 35
3.4 Éléments SNMP configurés pour le routeur VyOS-Router-CDEC 36
3.5 Liste des hôtes déclarés dans le serveur Zabbix de la CDEC 37
3.6 Confirmation de la création de l'action de notification. 38
3.7 Courriel de notification reçu suite au déclenchement du scénario de test 41
3.8 Tableau de bord Zabbix présentant plusieurs incidents de sécurité actifs 42

LISTE DES TABLEAUX

TABLEAU **Page**

1.1 Facteurs clés de l'environnement de supervision visé 8
1.2 Comparatif des versions du protocole SNMP 9
1.3 Correspondance entre les outils Kali Linux mobilisés et le MITRE ATT&CK 12
1.4 Comparatif des solutions de supervision par rapport au projet retenu 13

2.1 Limites de la supervision réseau actuelle de la CDEC 17
2.2 Exigences fonctionnelles du système de supervision proposé 18
2.3 Exigences non fonctionnelles du système de supervision proposé 18
2.4 Contraintes techniques et expérimentales du projet 19
2.5 Répartition des rôles des composants de l'architecture 21
2.6 Justification des choix technologiques face aux alternatives écartées 22
2.7 Plan d'adressage IP de l'architecture de simulation 23
2.8 Répartition des méthodes de supervision par équipement 23
2.9 Conception des scénarios de test de sécurité. 24
2.10 Identification des acteurs du système de supervision 25
2.11 Description textuelle du cas d'utilisation UC_01 – Traiter une alerte 26

3.1 Provisionnement des machines virtuelles du laboratoire 31
3.2 Résultats des tests de connectivité inter-hôtes 32
3.3 Déclaration des hôtes supervisés dans Zabbix 36
3.4 Paramètres de configuration du type de média Email 37
3.5 Configuration de l'action de notification des incidents de sécurité 38
3.6 Synthèse de la configuration de détection des trois scénarios de sécurité 39
3.7 Synthèse comparative des trois scénarios de test de sécurité 42
3.8 Estimation du temps moyen de détection et de notification par scénario. 43

INTRODUCTION GÉNÉRALE

La fiabilité des systèmes d'information constitue aujourd'hui un enjeu stratégique pour toute organisation, publique comme privée. Dans un contexte mondial où la continuité des services numériques conditionne directement la continuité des activités métiers, l'absence de visibilité sur l'état des infrastructures réseau et systèmes est reconnue comme un facteur de risque opérationnel majeur, justifiant la mise en place de dispositifs de supervision continue capables de détecter les anomalies avant qu'elles n'affectent les utilisateurs [1]. Cette exigence est d'autant plus forte dans le secteur financier, où la fiabilité, la disponibilité et la traçabilité des opérations sont à la fois des impératifs opérationnels et des obligations de fait vis-à-vis des usagers et des autorités de tutelle.

Au Cameroun, cette exigence se pose avec une acuité particulière pour les établissements publics financiers en cours de structuration. C'est le cas de la Caisse des Dépôts et Consignations du Cameroun (CDEC), établissement public de type particulier institué par la loi n°2008/003 du 14 avril 2008 régissant les dépôts et consignations [2] et organisé par le décret n°2011/105 du 15 avril 2011 [3], mais dont l'opérationnalisation effective n'est intervenue que le 20 janvier 2023, avec la nomination de ses premiers dirigeants après près de quinze années d'inactivité institutionnelle [4, 5]. Cette montée en puissance récente place la CDEC dans une phase active de structuration de ses systèmes d'information, phase au cours de laquelle les choix d'architecture technique retenus aujourd'hui conditionneront la robustesse opérationnelle de l'établissement pour les années à venir.

Contexte et justification

C'est dans ce contexte de structuration que s'inscrit le stage académique de fin d'études réalisé au sein de la Direction des Systèmes d'Information (DSI) de la CDEC, sous la supervision de M. FRANKO ATEKWANA. Les observations de terrain conduites en début de stage, en concertation avec l'équipe de la DSI, ont mis en évidence une infrastructure réseau et systèmes en croissance — postes de travail, serveurs applicatifs, équipements d'interconnexion et dispositifs de sécurité périmétrique — disposant déjà de plusieurs mécanismes de supervision, mais dont les capacités de centralisation et de visibilité peuvent être renforcées afin d'améliorer le suivi global de l'infrastructure. L'état de disponibilité des équipements et des services n'est, à ce jour, connu de la DSI que par signalement direct des utilisateurs, ce qui ne permet ni une détection précoce des incidents, ni une analyse de tendance sur les performances du parc informatique.

Cette situation n'est pas propre à la CDEC : elle caractérise plus largement les établissements

publics camerounais en phase de structuration de leur système d'information, pour lesquels la mise en place d'un dispositif de supervision représente à la fois un investissement technique mesuré – grâce à l'existence de solutions open source matures telles que Zabbix [6] – et un gain opérationnel immédiat en matière de continuité de service. C'est cette double réalité, technique et organisationnelle, qui justifie le choix du présent sujet de mémoire.

Problématique

Les approches actuellement mobilisées par la DSI de la CDEC pour assurer la disponibilité de son infrastructure reposent exclusivement sur une supervision manuelle et réactive : un incident n'est traité qu'après avoir été constaté et signalé par un utilisateur ou un agent. Cette approche présente plusieurs limites structurelles : l'absence de détection automatique des anomalies, l'absence d'historique exploitable pour anticiper les dégradations de performance, et une charge de diagnostic élevée pour une équipe DSI de taille réduite, contrainte de reconstituer manuellement le contexte de chaque incident. Aucun outil, méthode ou procédure formalisée ne permet aujourd'hui à la DSI de disposer d'une vue d'ensemble, en temps réel, de l'état de son parc informatique et réseau.

Question de recherche

Comment concevoir et valider une solution de supervision réseau et systèmes permettant à la DSI de la CDEC de détecter proactivement les incidents, tout en tenant compte de ses contraintes budgétaires et de ses compétences internes ?

Hypothèse de travail

Nous formulons l'hypothèse qu'une solution de supervision fondée sur l'outil open source Zabbix, dans sa version 7.0 LTS, combinant supervision par agent logiciel pour les postes et serveurs et supervision par le protocole normalisé SNMP [7] pour les équipements réseau actifs, permet de répondre à l'ensemble des exigences identifiées à la CDEC, à condition d'être précédée d'une validation complète en environnement de simulation avant tout déploiement sur l'infrastructure réelle. Deux aspects structurent cette proposition :

- **Couverture fonctionnelle unifiée :** en combinant, au sein d'un même outil gratuit, la supervision par agent des postes et serveurs et la supervision SNMP des équipements réseau, la solution proposée évite à la DSI de multiplier les outils et les compétences à mobiliser, tout en couvrant l'intégralité du périmètre technique identifié.
- **Démarche de validation préalable :** en imposant une phase de simulation complète (GNS3, VMware Workstation) avant toute intervention sur le réseau réel de production,

la démarche retenue élimine le risque opérationnel associé à un déploiement direct sur l'infrastructure d'un établissement financier en activité.

Objectifs

L'objectif général de ce travail est de concevoir et de valider une solution de supervision réseau et systèmes pour la CDEC, documentée de façon à permettre sa transposition ultérieure vers l'infrastructure réelle de production. Pour y parvenir, le travail se décline selon les axes suivants :

- Analyser l'infrastructure existante de la CDEC et les besoins de supervision exprimés par la DSI, et en déduire un cahier des charges fonctionnel et non fonctionnel précis.
- Etudier les solutions de supervision réseau disponibles sur le marché et justifier, sur des critères objectifs, le choix de Zabbix 7.0 LTS comme socle technologique du projet.
- Concevoir une architecture de supervision adaptée au contexte de la CDEC, et la valider intégralement dans un environnement de simulation reproduisant fidèlement la topologie réseau cible.
- Déployer, tester et documenter la solution retenue – y compris son mécanisme d'alerte automatique par courriel – de manière à produire un guide d'exploitation exploitable de façon autonome par la DSI.

Méthodologie adoptée

La démarche scientifique retenue pour ce mémoire repose sur un principe directeur : valider avant de déployer. Elle combine une analyse documentaire et normative (textes fondateurs de la CDEC, normes IETF relatives à SNMP, documentation officielle des outils retenus) avec une démarche d'ingénierie pratique en quatre temps : analyse de l'existant et spécification des exigences, conception de l'architecture cible et modélisation UML des interactions du système, validation expérimentale complète en environnement virtualisé (GNS3, VMware Workstation, VyOS, pfSense, Zabbix), puis analyse critique des résultats obtenus en vue de leur transposition vers l'infrastructure réelle de la CDEC. Cette démarche est conforme aux exigences d'un mémoire d'ingénieur de conception : elle privilégie la profondeur d'implémentation pratique et la reproductibilité des résultats plutôt qu'une contribution scientifique nouvelle.

Structure du mémoire

La suite de ce mémoire est structurée en trois chapitres :

- **Chapitre 1 : Concepts généraux et état de l'art**
  Ce chapitre définit les bases théoriques nécessaires à la compréhension du projet – supervision réseau, protocoles SNMP et agents, virtualisation et simulation réseau – et dresse un panorama comparatif des solutions de supervision existantes, afin de justifier le positionnement technologique retenu.
- **Chapitre 2 : Méthodologie**
  Cette partie présente la démarche méthodologique suivie, l'analyse de l'existant et des exigences, l'architecture globale de la solution, le plan d'adressage retenu ainsi que la modélisation UML des interactions entre la DSI et le système de supervision.
- **Chapitre 3 : Implémentation et résultats**
  Ce dernier chapitre présente la mise en œuvre technique détaillée de la solution en environnement de simulation, les scénarios de test conduits – y compris la simulation de comportements anormaux et d'attaques contrôlées destinée à valider le mécanisme d'alerte – ainsi que l'analyse critique des résultats obtenus et la démarche de transposition vers la production.

CHAPITRE 1

CONCEPTS GÉNÉRAUX ET ÉTAT DE L'ART

Ce chapitre pose les bases théoriques et techniques nécessaires à la compréhension du projet. Il définit les concepts fondamentaux de la supervision des systèmes d'information, détaille les protocoles, mécanismes et indicateurs sur lesquels elle repose, présente les outils de virtualisation, de simulation réseau et de test de sécurité mobilisés pour ce mémoire – en les situant par rapport au référentiel international MITRE ATT&CK –, puis dresse un panorama comparatif des solutions de supervision existantes afin de justifier le positionnement technologique retenu pour la Caisse des Dépôts et Consignations du Cameroun (CDEC).

Contents

1.1 Généralités sur la supervision réseau **6**
1.1.1 Définitions, finalités et approches de collecte 6
1.1.2 Architecture générique d'un système de supervision 6
1.1.3 L'environnement hétérogène visé : le cas de la CDEC 7
1.2 Protocoles et indicateurs de supervision **8**
1.2.1 Le protocole SNMP : architecture, MIB et versions 8
1.2.2 Supervision par agent logiciel et complémentarité avec SNMP 9
1.2.3 Indicateurs de fiabilité : MTBF, MTTR et disponibilité 10
1.3 Virtualisation, simulation et outils de test **10**
1.3.1 Virtualisation et simulation de topologies 10
1.3.2 Équipements réseau simulés : VyOS et pfSense 11
1.3.3 Kali Linux et cadre de référence MITRE ATT&CK 11
1.4 Panorama et positionnement des solutions de supervision **12**
1.4.1 Panorama comparatif des solutions existantes 12
1.4.2 Bilan et positionnement du projet 13

1.1 Généralités sur la supervision réseau

1.1.1 Définitions, finalités et approches de collecte

La supervision informatique, ou *monitoring*, désigne l'ensemble des techniques et outils permettant de surveiller en continu l'état, la disponibilité et les performances des ressources d'un système d'information : équipements réseau, serveurs, postes de travail et services applicatifs. Elle vise à donner à l'administrateur système une visibilité en temps réel sur l'infrastructure, afin de détecter les anomalies avant qu'elles n'affectent significativement les utilisateurs finaux. Le *National Institute of Standards and Technology* (NIST) souligne à ce titre que l'absence de visibilité continue sur l'état des actifs informatiques constitue un facteur de risque opérationnel majeur pour toute organisation [1].

On distingue classiquement trois finalités complémentaires : la **supervision de disponibilité**, qui vérifie qu'un équipement ou un service répond correctement (état *up*/*down*) ; la **supervision de performance**, qui mesure l'évolution de métriques dans le temps (charge processeur, utilisation mémoire, bande passante, latence) ; et la **supervision applicative**, qui contrôle le bon fonctionnement de services spécifiques (base de données, serveur web, messagerie). Dans le contexte d'un établissement financier public tel que la CDEC, ces trois finalités sont indissociables : une dégradation de performance non détectée à temps peut aboutir à une indisponibilité de service affectant directement la continuité des opérations métiers de dépôt et de consignation.

La collecte des données de supervision repose par ailleurs sur deux approches complémentaires. La **supervision active**, ou *polling*, consiste pour le serveur de supervision à interroger périodiquement les équipements surveillés afin d'en récupérer l'état ; c'est l'approche la plus répandue, simple à mettre en œuvre, mais dont la fréquence d'interrogation conditionne directement la réactivité de détection. La **supervision passive**, ou événementielle (*trap*), inverse cette logique : ce sont les équipements ou agents supervisés qui envoient spontanément une notification au serveur de supervision lors d'un changement d'état, réduisant ainsi la charge liée aux interrogations répétées. La solution retenue pour ce mémoire, présentée en section 1.4, combine ces deux logiques : interrogation active périodique pour la plupart des métriques système et réseau, et remontée quasi événementielle pour les éléments de journal utilisés dans les scénarios de sécurité du chapitre 3, dont l'intervalle de collecte a été réduit à 10 secondes précisément pour s'approcher d'un comportement événementiel.

1.1.2 Architecture générique d'un système de supervision

Une solution de supervision repose sur une architecture modulaire commune, indépendamment de l'éditeur retenu, résumée par la figure 1.1.

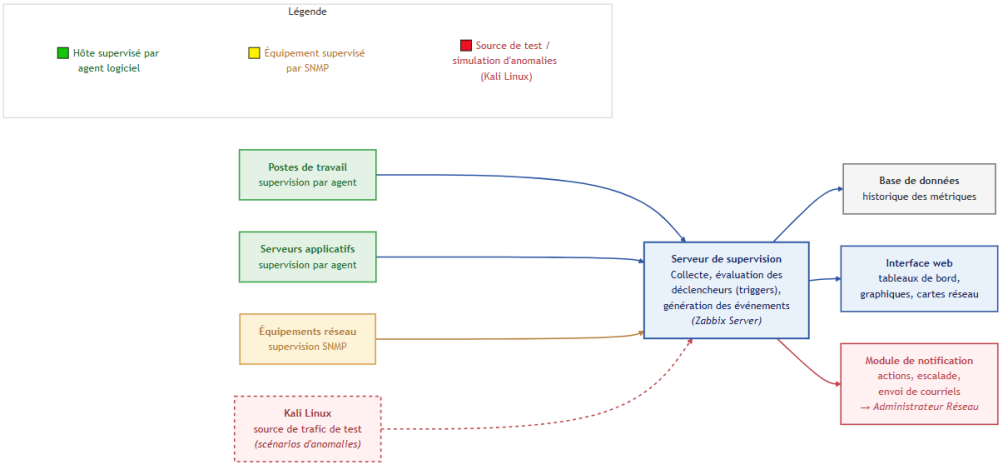


FIGURE 1.1 – Architecture générique d'un système de supervision réseau

Cette architecture distingue six composantes structurantes : un **serveur de supervision**, cœur du système, chargé de l'ordonnancement des collectes et de l'évaluation des règles d'alerte ; une **base de données**, qui conserve l'historique des métriques et permet l'analyse de tendance ; des **agents logiciels**, installés sur les équipements supervisés pour une collecte fine ; des **équipements sans agent**, interrogés à distance via un protocole normalisé tel que SNMP ; une **interface web**, pour la consultation par les administrateurs ; et un **module de notification**, qui alerte automatiquement les administrateurs en cas d'anomalie. Comme l'illustre également cette figure, un dispositif complémentaire de **source de test** (machine Kali Linux) a été intégré au périmètre de ce mémoire afin de générer des comportements anormaux contrôlés et de valider expérimentalement la chaîne de détection et de notification – démarche dont la justification méthodologique est développée en section 1.3.3 et la mise en œuvre détaillée au chapitre 3.

1.1.3 L'environnement hétérogène visé : le cas de la CDEC

L'infrastructure informatique et réseau de la CDEC, décrite plus en détail au chapitre 2, présente une hétérogénéité typique des établissements publics camerounais en phase de structuration : postes de travail, serveurs applicatifs et équipements réseau actifs de nature différente, sans historique de supervision centralisée préexistant sur lequel s'appuyer. Le tableau 1.1 synthétise les facteurs déterminants de cet environnement pour la conception de la solution de supervision, chacun étant directement répercuté sur une décision de conception prise au chapitre 2.

TABLE 1.1 – Facteurs clés de l'environnement de supervision visé

Facteur	Description	Impact sur la conception
Hétérogénéité des hôtes	Postes Linux/Windows, serveurs applicatifs, équipements réseau fermés	Nécessite de combiner supervision par agent et supervision SNMP
Criticité opérationnelle	Établissement financier public en activité	Impose une validation complète en environnement simulé avant tout déploiement réel
Compétences internes	Équipe DSI de taille réduite, sans expertise préalable en supervision	Oriente vers une solution à interface intégrée, sans surcouche logicielle
Contrainte budgétaire	Établissement en phase de structuration	Exclut les solutions à licence payante au-delà d'un seuil de capteurs

Ce tableau met en évidence que la contrainte la plus structurante n'est pas de nature purement technique, mais organisationnelle : c'est la combinaison de compétences internes limitées et d'une criticité opérationnelle élevée qui oriente, dès ce stade, vers une solution intégrée et validée par simulation plutôt que vers un assemblage d'outils spécialisés nécessitant une expertise avancée.

1.2 Protocoles et indicateurs de supervision

1.2.1 Le protocole SNMP : architecture, MIB et versions

Le protocole **SNMP** (*Simple Network Management Protocol*) constitue le standard historique de supervision des équipements réseau. Il repose sur une architecture normalisée par l'*Internet Engineering Task Force* (IETF), décrite dans la RFC 3411, qui définit un cadre modulaire conçu pour permettre l'évolution des normes du protocole dans le temps [7]. Cette architecture distingue les entités dotées d'applications répondeurs de commandes ou générateurs de notifications, traditionnellement appelées *agents SNMP*, des stations de gestion réseau dotées d'applications génératrices de commandes, appelées *managers* [8]. Les éléments techniques centraux de ce protocole sont la **MIB** (*Management Information Base*), base structurée et normalisée décrivant l'ensemble des variables interrogeables sur un équipement [9], et les **OID** (*Object Identifiers*), identifiants hiérarchiques uniques associés à chaque variable de la MIB. Un OID s'exprime sous forme d'une suite numérique séparée par des points, par exemple pour le temps de fonctionnement d'un équipement :

(1.1) `1.3.6.1.2.1.1.3.0` ⟶ *sysUpTime*

Cet OID particulier, `sysUpTime`, est directement réutilisé au chapitre 3 pour la supervision SNMP du routeur VyOS et du pare-feu pfSense, en l'absence de modèle officiel Zabbix dédié à

ces équipements.

Le tableau 1.2 compare les trois versions du protocole selon leurs mécanismes d'authentification et de chiffrement.

TABLE 1.2 – Comparatif des versions du protocole SNMP

Critère	**SNMPv1**	**SNMPv2c**	**SNMPv3**
Authentification	Communauté en clair	Communauté en clair	Utilisateur (USM)
Chiffrement	Aucun	Aucun	AES / DES [10]
Complexité de configuration	Faible	Faible	Élevée
Usage recommandé	Obsolète	Environnements internes non critiques	Environnements sensibles

Ce comparatif justifie directement un arbitrage assumé dans ce mémoire : la version 3 introduit le *User-based Security Model* (USM), apportant authentification et chiffrement des échanges [10], mais au prix d'une complexité de configuration nettement supérieure. Dans le laboratoire de simulation, SNMPv2c a été retenu à des fins de validation fonctionnelle – l'objectif prioritaire étant de valider l'architecture et la chaîne d'alerte plutôt que de complexifier prématurément la configuration – la migration vers SNMPv3 étant explicitement identifiée comme recommandation de sécurisation avant tout déploiement en production (chapitre 2, section 2.8).

1.2.2 Supervision par agent logiciel et complémentarité avec SNMP

Contrairement à SNMP, un **agent de supervision** est un logiciel installé localement sur la machine supervisée. Il collecte des informations détaillées – processus, services, journaux, ressources matérielles – et les transmet au serveur central selon un mode actif (l'agent initie l'envoi) ou passif (le serveur interroge l'agent) [6]. Cette approche offre une granularité d'information nettement plus fine que SNMP : c'est elle, notamment, qui rend possible la surveillance de contenu de journaux systèmes (*log monitoring*) mobilisée pour les trois scénarios de sécurité du chapitre 3, un niveau de détail hors de portée d'une simple interrogation SNMP. Elle nécessite en contrepartie l'installation d'un logiciel sur chaque machine, ce qui n'est pas toujours possible pour des équipements réseau fermés.

Une infrastructure hétérogène, telle que celle visée à la CDEC, combine donc nécessairement les deux approches : agent logiciel pour les postes et serveurs ouverts, SNMP pour les équipements réseau fermés (routeur, pare-feu). Cette complémentarité, déjà identifiée dans la littérature comparative sur les outils de supervision open source [**squ_comparative**], structure directement l'architecture de supervision détaillée au chapitre 2 (tableau 2.2.8).

1.2.3 Indicateurs de fiabilité : MTBF, MTTR et disponibilité

Deux indicateurs classiques de la gestion des services informatiques, formalisés notamment dans le référentiel *ITIL* (*Information Technology Infrastructure Library*) et sa norme associée ISO/CEI 20000 [**itil_wikipedia**], permettent d'objectiver l'apport d'une supervision proactive : le temps moyen entre pannes (MTBF) et le temps moyen de réparation (MTTR), respectivement définis par :

$$\text{MTBF} = \frac{\text{Temps total de fonctionnement}}{\text{Nombre de pannes}} \tag{1.2}$$

$$\text{MTTR} = \frac{\text{Temps total de réparation}}{\text{Nombre d'incidents résolus}} \tag{1.3}$$

Ces deux grandeurs permettent de calculer la disponibilité théorique d'un système :

$$\text{Disponibilité} = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} \tag{1.4}$$

L'apport attendu d'une supervision proactive réside précisément dans la réduction du MTTR : la détection automatique et immédiate d'un incident raccourcit le délai entre son apparition et son traitement, comparativement à une détection par signalement utilisateur. Le référentiel ITIL distingue par ailleurs plus finement le MTTR (temps de réparation d'un composant) du MTRS – *Mean Time to Restore Service* – qui mesure le temps de rétablissement du service dans son ensemble [**itil_wikipedia**] ; cette nuance, bien que non exploitée dans les mesures du chapitre 3 pour des raisons de simplicité, souligne qu'un MTTR technique faible ne garantit pas à lui seul un rétablissement rapide du service perçu par l'utilisateur final. Ces indicateurs seront repris au chapitre 3 pour évaluer, de façon quantitative, les résultats des scénarios de détection mis en œuvre.

1.3 Virtualisation, simulation et outils de test

1.3.1 Virtualisation et simulation de topologies

La virtualisation consiste à exécuter, sur une même machine physique, plusieurs systèmes d'exploitation indépendants au moyen d'un **hyperviseur**. L'hyperviseur **VMware Workstation** a été retenu pour héberger les machines virtuelles supervisées, en raison de sa gestion avancée des réseaux virtuels (*VMnet*) et de sa compatibilité avec la machine d'exécution de GNS3 [11]. **GNS3** (*Graphical Network Simulator 3*), pour sa part, permet de concevoir, câbler et exécuter des topologies réseau complexes en s'appuyant sur des images réelles de systèmes d'exploitation réseau plutôt que sur de simples modèles simplifiés [12]. Le pont entre la topologie simulée dans

GNS3 et des machines virtuelles indépendantes s'effectue via des *Cloud nodes*, associés à des interfaces réseau virtuelles de l'hyperviseur – principe technique central de l'architecture de laboratoire présentée au chapitre 2 et mis en œuvre au chapitre 3.

1.3.2 Équipements réseau simulés : VyOS et pfSense

VyOS est un système d'exploitation réseau open source, dérivé de Debian, fournissant les fonctions d'un routeur et d'un pare-feu logiciels : routage, NAT, VPN et services réseau standards tels que SNMP [13]. **pfSense** est, quant à lui, une distribution pare-feu et routeur open source fondée sur FreeBSD, proposant une interface de gestion web complète pour la configuration du filtrage, du NAT et de services complémentaires tels que SNMP [14]. Le choix de combiner ces deux composants plutôt que de confier l'ensemble des fonctions à VyOS seul – envisagé puis écarté, comme détaillé au chapitre 2 (tableau 2.2.6) – repose sur la spécialisation de pfSense pour la configuration fine du filtrage périmétrique, via une interface dédiée mieux adaptée à une équipe DSI sans expertise préalable en administration de pare-feu en ligne de commande.

1.3.3 Kali Linux et cadre de référence MITRE ATT&CK

Afin de valider expérimentalement le mécanisme de détection et d'alerte de la solution de supervision – et non uniquement sa capacité à afficher un état statique –, ce mémoire intègre une machine **Kali Linux**, distribution GNU/Linux orientée vers les tests de sécurité et l'audit réseau, préinstallée avec un ensemble d'outils standards d'administration et de test de sécurité [15]. Trois outils de cette distribution sont mobilisés au chapitre 3 pour générer des comportements réseau anormaux contrôlés, détectés par les déclencheurs Zabbix configurés à cet effet : **Nmap**, outil de cartographie et de scan de ports réseau, utilisé pour générer un scan de reconnaissance contre un hôte supervisé [16] ; **Hydra**, outil de test d'authentification par force brute, utilisé pour générer des tentatives de connexion SSH répétées [17] ; et **hping3**, outil de génération de paquets réseau personnalisés, utilisé pour produire une rafale de trafic simulant une saturation de lien réseau [18].

Afin d'ancrer ces trois comportements dans un cadre de référence reconnu plutôt que de les présenter comme de simples manipulations *ad hoc*, le tableau 1.3 positionne chacun d'eux par rapport à la matrice **MITRE ATT&CK**, référentiel international de classification des tactiques et techniques d'attaque largement adopté par l'industrie de la cybersécurité et les équipes de détection (*Security Operations Center*) [**mitre_t1046**, **mitre_t1110**, **mitre_t1498**].

TABLE 1.3 – Correspondance entre les outils Kali Linux mobilisés et le MITRE ATT&CK

Outil	Tactique ATT&CK	Technique	Principe
Nmap	*Discovery* (reconnaissance)	T1046 – *Network Service Discovery*	Identification des services actifs sur les hôtes distants par scan de ports [**mitre_t1046**]
Hydra	*Credential Access* (accès aux identifiants)	T1110 – *Brute Force*	Découverte d'un mot de passe par soumission répétée de combinaisons [**mitre_t1110**]
hping3	*Impact*	T1498 – *Network Denial of Service*	Saturation d'un lien ou d'un service par un volume de trafic anormal [**mitre_t1498**]

Ce positionnement présente un double intérêt méthodologique. D'une part, il montre que les trois scénarios retenus ne sont pas arbitraires mais couvrent trois tactiques *distinctes* de la matrice ATT&CK (reconnaissance, accès aux identifiants, impact), offrant une validation plus représentative de la chaîne de détection que trois variantes d'un même type d'attaque. D'autre part, il permet, au chapitre 3, d'évaluer objectivement une limite du dispositif : l'absence de système de détection d'intrusion (IDS) dédié restreint la détection aux techniques dont l'exécution produit une trace mesurable dans les journaux ou les métriques déjà collectées par l'agent Zabbix, ce qui n'est pas garanti pour l'ensemble des techniques répertoriées dans la matrice ATT&CK. Cette démarche, courante dans la validation des dispositifs de supervision et de détection d'incidents, permet de vérifier que la chaîne complète – collecte, évaluation du déclencheur, génération de l'événement, notification par courriel à l'administrateur DSI – fonctionne effectivement de bout en bout, et non uniquement sur le papier.

1.4 Panorama et positionnement des solutions de supervision

1.4.1 Panorama comparatif des solutions existantes

Le marché des solutions de supervision réseau, qu'il s'agisse de logiciels libres ou propriétaires, reste structuré autour d'un nombre restreint d'acteurs matures, dont les forces et faiblesses respectives sont documentées tant par la littérature académique [**squ_comparative**] que par les comparatifs professionnels du secteur [**netadmintools2023**]. **Nagios** est l'une des solutions de supervision open source les plus anciennes et les plus répandues, reposant sur un moteur de vérification par *plugins* et une architecture modulaire extensible [19] ; sa configuration, essentiellement fondée sur des fichiers texte, exige une expertise préalable, et son interface web native reste sommaire sans surcouche complémentaire. **Centreon** est construit sur le moteur Nagios/Centreon Engine, auquel il ajoute une interface web moderne et des fonctionnalités de gestion de configuration facilitées [20] ; sa richesse fonctionnelle en fait une solution puissante,

mais parfois complexe à déployer pour une équipe de taille réduite. **PRTG Network Monitor**, solution propriétaire de l'éditeur Paessler, est reconnue pour la simplicité de sa prise en main [21], mais son modèle de licence, fondé sur le nombre de capteurs, la rend rapidement coûteuse au-delà d'un certain seuil, et son caractère propriétaire limite la personnalisation. **Cacti**, enfin, est un outil historiquement orienté vers la représentation graphique de données de performance collectées via SNMP [22], peu adapté à la gestion d'alertes complexes ou à la supervision applicative fine.

Zabbix, quatrième grande alternative open source, se distingue en combinant nativement supervision par agent, supervision SNMP, moteur de gestion d'alertes flexible et interface web complète, au sein d'une distribution en version *Long Term Support* (LTS) [6]. Les comparatifs techniques récents confirment cette position : Zabbix propose une interface web moderne et une configuration graphique complète prête à l'emploi, tandis que Nagios, en contrepartie d'une richesse de plus de 5000 *plugins* communautaires, impose une configuration essentiellement textuelle et une courbe d'apprentissage plus abrupte [**netadmintools2023**]. Le tableau 1.4 synthétise ce positionnement sur les critères directement pertinents pour le cahier des charges de la DSI de la CDEC établi au chapitre 2.

TABLE 1.4 – Comparatif des solutions de supervision par rapport au projet retenu

Critère	Nagios	Centreon	PRTG	Cacti	ZABBIX (retenu)
Gratuité	✓	—	✗	✓	✓✓
Supervision agent native	—	—	✓	✗	✓✓
Supervision SNMP	✓	✓	✓	✓	✓✓
Interface web intégrée	✗	✓	✓	—	✓✓
Version LTS	—	—	—	—	✓✓
Simplicité de prise en main	—	—	✓	—	✓

Ce tableau fait apparaître que Zabbix n'obtient pas la note maximale sur l'ensemble des critères – PRTG et, dans une moindre mesure, Centreon restent perçus comme plus immédiatement accessibles à la prise en main – mais qu'il est la seule solution à combiner un score *très bon* sur les quatre premiers critères techniques (gratuité, agent natif, SNMP, interface intégrée), sans le compromis de licence qu'impose PRTG. C'est cette combinaison, plutôt qu'une supériorité isolée sur un seul critère, qui motive le choix retenu.

1.4.2 Bilan et positionnement du projet

L'analyse conduite dans ce chapitre met en évidence trois constats structurants. Premièrement, la supervision réseau moderne repose sur une combinaison indissociable de protocoles normalisés (SNMP) et de mécanismes de collecte par agent, chacun couvrant un périmètre d'équipements différent – constat directement applicable à l'infrastructure hétérogène de la CDEC. Deuxièmement, le panorama des solutions existantes révèle un compromis récurrent entre ri-

chesse fonctionnelle, coût de licence et complexité de prise en main, sans solution dominant simultanément ces trois dimensions. Troisièmement, la validation d'un mécanisme d'alerte ne peut se limiter à une vérification déclarative de sa configuration : elle nécessite une mise à l'épreuve par des comportements anormaux contrôlés, rattachables à un référentiel reconnu tel que MITRE ATT&CK, faute de quoi rien ne garantit que la chaîne de notification fonctionne effectivement au moment où elle serait sollicitée par un incident réel.

Zabbix se distingue par la combinaison, en une seule solution gratuite, d'une couverture fonctionnelle complète, d'une interface web intégrée et d'une distribution LTS garantissant un support de sécurité prolongé – critère déterminant pour un système destiné à demeurer en production plusieurs années après la fin du stage à l'origine de ce mémoire. Ce positionnement, mis en regard du cahier des charges de la DSI de la CDEC, justifie le choix de Zabbix 7.0 LTS comme socle technologique de la solution proposée, combiné aux outils de virtualisation et de simulation présentés en section 1.3 – y compris la machine Kali Linux mobilisée au chapitre 3 pour valider expérimentalement, selon le cadre MITRE ATT&CK introduit en section 1.3.3, le mécanisme d'alerte de la solution. La conception détaillée de cette solution fait l'objet du chapitre 2.

CHAPITRE 2

MÉTHODOLOGIE

Ce chapitre présente la démarche méthodologique adoptée pour la conception de la solution de supervision, l'analyse de l'existant conduite au sein de la Direction des Systèmes d'Information (DSI) de la CDEC, la spécification des exigences, l'architecture globale retenue – intégrant un poste de test de sécurité sous Kali Linux –, le plan d'adressage, la conception détaillée de la supervision par type d'équipement et des scénarios de test associés, ainsi que la modélisation UML de la solution et la démarche de transposition vers la production.

Contents

2.1 Démarche méthodologique et analyse de l'existant **16**
2.1.1 Principe général et étapes de la démarche 16
2.1.2 Analyse de l'existant et limites de la supervision actuelle 16
2.2 Spécification des exigences du système **17**
2.2.1 Exigences fonctionnelles et non fonctionnelles 17
2.2.2 Contraintes techniques et expérimentales 19
2.3 Architecture globale de la solution proposée **19**
2.3.1 Environnement hybride et intégration du poste de test Kali Linux . . . 19
2.3.2 Topologie, répartition des rôles et justification des choix technologiques 20
2.4 Plan d'adressage et conception de la supervision par équipement **22**
2.4.1 Plan d'adressage et segmentation réseau 22
2.4.2 Répartition des méthodes de supervision et organisation des hôtes . . . 23
2.4.3 Mécanisme d'alerte et scénarios de test de sécurité 24
2.5 Modélisation UML de la solution **25**
2.5.1 Acteurs et diagramme de cas d'utilisation 25
2.5.2 Diagramme de séquence : cycle de détection et de notification 26
2.6 Démarche de transposition vers la production **27**
Bilan du chapitre **28**

2.1 Démarche méthodologique et analyse de l'existant

2.1.1 Principe général et étapes de la démarche

Compte tenu de la criticité opérationnelle d'un établissement financier public, toute intervention directe sur l'infrastructure réseau réelle de la CDEC devait être précédée d'une phase de validation ne présentant aucun risque pour la continuité de service. La démarche méthodologique retenue s'articule donc autour d'un principe directeur : **concevoir, câbler et valider intégralement l'architecture de supervision dans un environnement de simulation fidèle, avant toute reproduction sur l'infrastructure réelle**. Cette logique rejoint les principes de gestion continue de la supervision des systèmes d'information formalisés par le NIST, selon lesquels la supervision continue doit reposer sur une stratégie clairement définie avant sa mise en œuvre opérationnelle [1]. Ce principe n'est pas une simple précaution méthodologique : il découle directement du constat, établi au chapitre 1, qu'une validation purement déclarative d'un mécanisme d'alerte – sans mise à l'épreuve par des comportements anormaux effectifs – ne garantit en rien son fonctionnement au moment où il serait sollicité par un incident réel.

Cette démarche comprend six étapes séquentielles, résumées par la figure 2.1 : analyse de l'existant, spécification des exigences, conception de l'architecture cible, modélisation UML, validation en environnement simulé, et transposition vers la production.

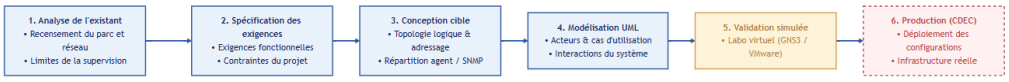


FIGURE 2.1 – Démarche méthodologique adoptée pour la conception de la solution

Ce mémoire couvre intégralement les étapes 1 à 5 ; l'étape 6, relative au déploiement en production, est planifiée pour la suite du projet et discutée en section 2.6. Le choix de circonscrire le périmètre expérimental à un environnement de simulation, plutôt que de procéder par ajustements successifs directement sur l'infrastructure réelle, découle directement du principe énoncé ci-dessus : chaque étape ne devient un prérequis à la suivante qu'une fois entièrement validée, ce qui explique la structure strictement séquentielle – et non itérative – de la figure 2.1.

2.1.2 Analyse de l'existant et limites de la supervision actuelle

L'analyse de l'existant a été conduite en collaboration avec l'encadreur professionnel, M. NANGUH Franklin, et l'équipe de la DSI. Elle a permis d'identifier le périmètre technique retenu comme cible de supervision pour ce projet : des postes de travail sous environnement Linux et Windows, des serveurs internes assurant les services applicatifs de la DSI, un routeur assurant l'interconnexion avec les réseaux externes, et un pare-feu assurant la protection périmétrique du réseau interne.

(Section à compléter avec les données précises de l'inventaire DSI, constituant une source primaire citée dans la bibliographie comme CDEC-DSI, Inventaire du parc informatique, document interne, 2026.)

Cette analyse a également permis de caractériser, non pas l'absence de supervision, mais la nature exclusivement *réactive* de la pratique actuelle : tout incident n'est constaté qu'après signalement par un utilisateur ou un agent, jamais détecté de façon autonome. Le tableau 2.1 synthétise les cinq limites structurelles qui en découlent, chacune motivant directement une exigence fonctionnelle ou non fonctionnelle formulée en section 2.2.

TABLE 2.1 – Limites de la supervision réseau actuelle de la CDEC

Limite constatée	**Conséquence opérationnelle**
Absence d'outil centralisé de supervision	Aucune vue d'ensemble de l'état du parc informatique et réseau en temps réel
Détection réactive des incidents	Les pannes sont signalées par les utilisateurs, non détectées automatiquement
Absence d'historique de disponibilité	Impossibilité d'analyser les tendances de performance ou de planifier une maintenance préventive
Absence de mécanisme d'alerte automatique	Délai de réaction de la DSI allongé, augmentant le temps moyen de résolution (MTTR)
Absence de test de la chaîne d'alerte	Aucune garantie que les alertes, si elles existaient, parviendraient effectivement à l'administrateur

La dernière ligne de ce tableau mérite une attention particulière : elle anticipe un constat que la seule analyse de l'existant ne peut révéler à ce stade, mais que la littérature sur la gestion des services informatiques identifie comme une défaillance fréquente y compris dans des organisations dotées d'un outil de supervision – l'absence de validation active de la chaîne de notification. C'est ce constat, appliqué par anticipation à la solution encore à concevoir, qui justifie l'exigence EF-07 formulée en section 2.2 et l'intégration d'un poste de test dédié dès la phase de conception de l'architecture (section 2.3), plutôt que son ajout tardif après un premier déploiement.

2.2 Spécification des exigences du système

2.2.1 Exigences fonctionnelles et non fonctionnelles

À partir de l'analyse de l'existant et des échanges conduits avec la DSI, un cahier des charges fonctionnel a été formalisé en début de stage. Les sept exigences fonctionnelles du tableau 2.2 couvrent l'ensemble des finalités de supervision identifiées au chapitre 1 (disponibilité, performance, alerte automatique) et introduisent, avec EF-07, une exigence directement issue de la

limite structurelle identifiée ci-dessus : nulle part ailleurs dans le cahier des charges il n'est demandé de *prouver* qu'une fonctionnalité marche, sauf pour le mécanisme d'alerte – une dissymétrie assumée, la notification étant le seul maillon de la chaîne dont la défaillance silencieuse serait invisible tant qu'aucun incident réel ne survient.

TABLE 2.2 – Exigences fonctionnelles du système de supervision proposé

Réf.	**Exigence fonctionnelle**
EF-01	Superviser la disponibilité (*up*/*down*) des postes, serveurs et équipements réseau actifs
EF-02	Superviser les ressources systèmes (CPU, mémoire, disque, réseau) des postes et serveurs via un agent logiciel
EF-03	Superviser les équipements actifs (routeur, pare-feu) via le protocole SNMP
EF-04	Générer des alertes automatiques (*triggers*) en cas de dépassement de seuil ou d'indisponibilité
EF-05	Mettre à disposition des tableaux de bord consultables par la DSI, présentant les incidents actifs classés par sévérité
EF-06	Conserver un historique exploitable des métriques collectées
EF-07	**Valider expérimentalement le mécanisme d'alerte** au moyen de scénarios de comportements anormaux contrôlés, générés depuis un poste de test dédié

Les exigences non fonctionnelles du tableau 2.3 traduisent, quant à elles, les contraintes organisationnelles identifiées au chapitre 1 (tableau 1.1.1) : la contrainte budgétaire se retrouve dans ENF-01, la limitation des compétences internes dans ENF-02 et ENF-05, et la criticité opérationnelle de l'établissement dans ENF-04. L'exigence ENF-06, en miroir direct de EF-07, borne explicitement le périmètre du poste de test : sa légitimité en phase de validation ne l'autorise en aucun cas à subsister au-delà du laboratoire.

TABLE 2.3 – Exigences non fonctionnelles du système de supervision proposé

Réf.	**Exigence non fonctionnelle**
ENF-01	Solution libre et sans coût de licence
ENF-02	Installation manuelle des composants, pour une maîtrise complète par l'équipe DSI
ENF-03	Version *Long Term Support*, garantissant la maintenance et les correctifs de sécurité
ENF-04	Validation complète en environnement de simulation avant toute intervention sur l'infrastructure réelle
ENF-05	Documentation exhaustive, en langue française, permettant l'exploitation autonome par la DSI
ENF-06	Le poste de test de sécurité ne doit en aucun cas être répliqué en production ; il est strictement réservé à l'environnement de laboratoire

2.2.2 Contraintes techniques et expérimentales

Au-delà des exigences fonctionnelles et non fonctionnelles, cinq contraintes concrètes, résumées au tableau 2.4, ont borné l'ensemble des choix de conception qui suivent. La contrainte de délai (neuf semaines) explique le choix, discuté en section 2.3, de retenir des composants matures et documentés officiellement plutôt que des alternatives nécessitant un temps d'appropriation plus long ; la contrainte de matériel de simulation (une seule machine hôte) impose le recours à la virtualisation imbriquée détaillée au chapitre 1 ; et la contrainte d'isolement du poste de test découle directement de l'exigence ENF-06.

TABLE 2.4 – Contraintes techniques et expérimentales du projet

Contrainte	Description
Délai	Neuf semaines de stage, incluant analyse, conception, simulation et transposition vers la production
Infrastructure	Aucune intervention directe sur le réseau de production avant validation complète en simulation
Isolement du poste de test	Le poste Kali Linux doit rester strictement confiné au réseau de laboratoire, sans aucune interface vers le réseau réel de la CDEC
Compétences	Solution exploitable par l'équipe DSI existante, sans expertise préalable en supervision
Portée académique	Travail d'ingénieur de conception : profondeur d'implémentation pratique, sans contribution scientifique nouvelle

2.3 Architecture globale de la solution proposée

2.3.1 Environnement hybride et intégration du poste de test Kali Linux

La validation de l'architecture repose sur un environnement hybride combinant **GNS3**, pour la simulation de la topologie réseau, et **VMware Workstation**, pour l'hébergement des machines virtuelles supervisées. Le principe fondamental de cette architecture est que GNS3 et VMware Workstation constituent deux environnements d'exécution distincts, reliés par des interfaces réseau virtuelles partagées (*Cloud nodes* GNS3 associés à des réseaux *VMnet* VMware) – choix technique justifié au chapitre 1 par la nécessité de faire cohabiter des images réseau réelles (VyOS, pfSense) avec des systèmes d'exploitation complets (Ubuntu, Windows) sans recourir à un émulateur unique couvrant artificiellement les deux besoins.

Conformément à l'exigence EF-07, cette architecture intègre, en plus des trois hôtes supervisés, un quatrième poste virtuel sous distribution **Kali Linux** [15], dont le rôle diffère fondamentalement des autres : il n'est **pas** un actif à superviser, mais un **générateur de comportements anormaux contrôlés**, destiné à valider expérimentalement que la chaîne complète de détection et de notification – de la collecte de la métrique jusqu'à la réception du courriel d'alerte par l'administrateur DSI – fonctionne effectivement. Ce poste est positionné sur le même segment

réseau interne que les hôtes supervisés (`192.168.47.0/24`), plutôt que du côté externe du pare-feu. Ce choix reproduit un scénario réaliste et fréquemment documenté en administration de systèmes : celui d'un poste compromis ou d'une menace interne opérant déjà à l'intérieur du périmètre protégé, scénario contre lequel un pare-feu périmétrique seul n'offre aucune protection et pour lequel seule une supervision active des hôtes et des journaux permet une détection – argument renforcé au chapitre 1 (section 1.3.3) par le positionnement des trois outils mobilisés (Nmap, Hydra, hping3) sur trois tactiques distinctes de la matrice MITRE ATT&CK. Les scénarios de test précis exploitant ce poste sont détaillés en section 2.4.3 et mis en œuvre au chapitre 3.

2.3.2 Topologie, répartition des rôles et justification des choix technologiques

La figure 2.2 présente la topologie logique cible retenue, intégrant le poste Kali Linux au sein du réseau interne, en aval du pare-feu périmétrique – traduction visuelle directe du positionnement argumenté ci-dessus.

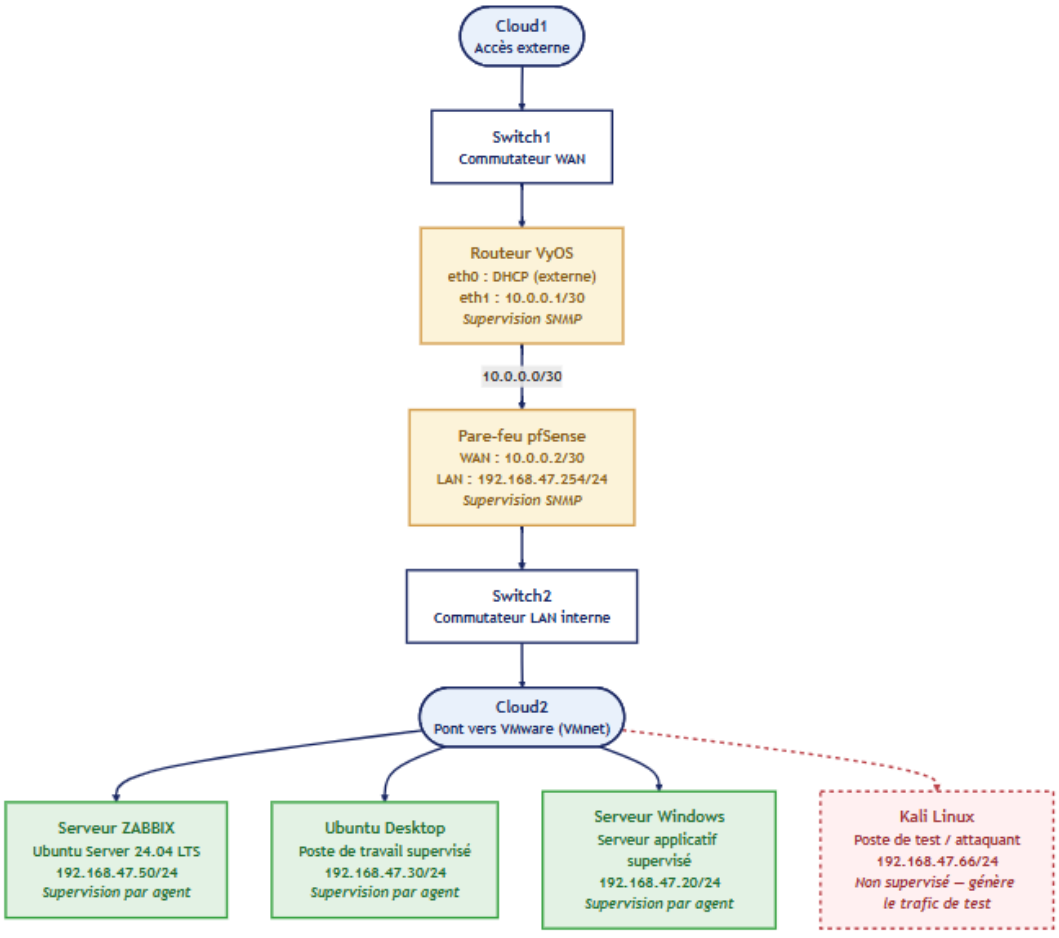


FIGURE 2.2 – Topologie logique cible de l'architecture de supervision.

Le tableau 2.5 détaille le rôle et la nature de chacun des six composants de cette topologie. Il fait apparaître une distinction structurante, également reflétée au tableau 2.7 et à la section 2.4 : cinq composants sont des *actifs* au sens de la supervision (à protéger et surveiller), tandis que le sixième – Kali Linux – occupe un statut délibérément différent, celui d'un instrument de validation.

TABLE 2.5 – Répartition des rôles des composants de l'architecture

Composant	**Rôle**	**Nature**
Routeur d'interconnexion	Interconnexion et NAT vers le réseau externe	VyOS (émulé sous GNS3)
Pare-feu périmétrique	Filtrage et protection du réseau interne	pfSense (émulé sous GNS3)
Serveur de supervision	Collecte, stockage et visualisation	Zabbix 7.0 LTS sur Ubuntu Server 24.04 (VM VMware)
Poste de travail supervisé	Représentatif d'un poste utilisateur Linux, cible principale des scénarios de test de sécurité	Ubuntu Desktop (VM VMware)
Serveur supervisé	Représentatif d'un serveur applicatif Windows	Windows Server (VM VMware)
Poste de test de sécurité	**Génération de comportements anormaux contrôlés (scan, brute force, rafale réseau)**	**Kali Linux (VM VMware, non supervisée)**

Chacun de ces six choix technologiques a été arrêté par comparaison explicite à une alternative plausible, synthétisée au tableau 2.6. Cette mise en regard systématique vise à écarter tout choix par défaut ou par simple familiarité : chaque ligne engage un critère objectif – support d'images réelles, gestion des réseaux virtuels, absence de contrainte de licence, richesse d'une interface de gestion dédiée, ou couverture fonctionnelle – directement traçable à une exigence des tableaux 2.2 ou 2.3.

TABLE 2.6 – Justification des choix technologiques face aux alternatives écartées

Composant retenu	Alternative écartée	Justification
GNS3	Cisco Packet Tracer, EVE-NG	Support natif d'images réelles et intégration avec des hyperviseurs externes [12]
VMware Workstation	VirtualBox	Meilleure gestion des réseaux virtuels multiples (VMnet) [11]
VyOS	Cisco IOS propriétaire	Solution open source, sans contrainte de licence [13]
pfSense	VyOS en pare-feu unique	Interface de gestion web dédiée, configuration fine du filtrage et du SNMP [14]
Zabbix 7.0 LTS	Nagios, Centreon, PRTG, Cacti	Couverture fonctionnelle complète, gratuité et support LTS (cf. chapitre 1, tableau 1.1.4)
Kali Linux	Machine Windows avec outils isolés	Distribution unique regroupant Nmap, Hydra et hping3, maintenue et documentée officiellement [15]

2.4 Plan d'adressage et conception de la supervision par équipement

2.4.1 Plan d'adressage et segmentation réseau

Le tableau 2.7 présente le plan d'adressage retenu, conçu pour être directement transposable à l'infrastructure réelle de la CDEC, à l'exception du poste Kali Linux qui, conformément à l'exigence ENF-06, ne doit exister qu'en environnement de laboratoire. Deux principes structurent ce plan : d'une part, une segmentation stricte entre la liaison point-à-point routeur/pare-feu (`10.0.0.0/30`) et le réseau interne (`192.168.47.0/24`), garantissant qu'aucun hôte interne n'est directement exposé au segment externe ; d'autre part, une adresse dédiée pour le poste Kali Linux au sein même du réseau interne, cohérente avec le positionnement argumenté en section 2.3.

TABLE 2.7 – Plan d'adressage IP de l'architecture de simulation

Équipement	Interface	Adresse IP	Rôle
Routeur VyOS	eth0	Dynamique (DHCP)	Interface externe
Routeur VyOS	eth1	10.0.0.1/30	Liaison point-à-point vers pfSense
Pare-feu pfSense	WAN	10.0.0.2/30	Liaison point-à-point vers VyOS
Pare-feu pfSense	LAN	192.168.47.254/24	Passerelle du réseau interne
Serveur Zabbix	–	192.168.47.50/24	Serveur de supervision
Poste Ubuntu Desktop	–	192.168.47.30/24	Hôte supervisé (agent), cible principale des scénarios de test
Serveur Windows	–	192.168.47.20/24	Hôte supervisé (agent)
Kali Linux	–	**192.168.47.66/24**	**Poste de test, non supervisé, laboratoire uniquement**

2.4.2 Répartition des méthodes de supervision et organisation des hôtes

Conformément à la complémentarité SNMP/agent établie au chapitre 1, chaque équipement du périmètre est rattaché à la méthode de supervision adaptée à sa nature (tableau 2.8) : agent logiciel pour les systèmes ouverts, SNMP pour les équipements réseau fermés. Le cas du poste Kali Linux illustre, par contraste, la logique retenue : n'étant ni un actif de production ni un équipement dont la disponibilité conditionne un service métier, il n'entre dans aucune des deux catégories et n'est, par construction, soumis à aucune méthode de supervision.

TABLE 2.8 – Répartition des méthodes de supervision par équipement

Équipement	Méthode	Justification
Serveur Zabbix (local)	Agent	Auto-supervision du serveur hôte
Poste Ubuntu Desktop	Agent (Zabbix Agent2)	Système ouvert, installation d'agent possible
Serveur Windows	Agent (Zabbix Agent2)	Système ouvert, installation d'agent possible
Routeur VyOS	SNMP	Équipement réseau fermé
Pare-feu pfSense	SNMP	Équipement réseau fermé, interface de gestion web dédiée
Kali Linux	**Aucune (non supervisé)**	**Représente une source de menace simulée, non un actif protégé**

Les hôtes réellement supervisés sont par ailleurs organisés en trois groupes logiques Zabbix – **CDEC – Postes**, **CDEC – Serveurs** et **CDEC – Réseau** –, reflétant leur nature fonctionnelle plutôt que leur seule localisation physique. Le poste Kali Linux n'appartient délibérément à aucun de ces groupes : il n'est pas déclaré comme hôte dans Zabbix, afin d'éviter toute ambiguïté entre hôte supervisé et source de test – une distinction qui, bien qu'apparemment secondaire, évite qu'un futur administrateur DSI n'interprète à tort la présence de Kali Linux dans l'inventaire Zabbix comme une invitation à le conserver en production.

2.4.3 Mécanisme d'alerte et scénarios de test de sécurité

Le mécanisme d'alerte repose sur le modèle standard de Zabbix – déclencheurs (*triggers*), actions et opérations de notification [6] – avec une graduation de sévérité en quatre niveaux (*Information*, *Warning*, *Average*, *High*). Le canal de notification retenu pour ce projet est le courrier électronique, adressé à l'administrateur DSI ; sa configuration détaillée, ainsi que celle des déclencheurs associés à chaque scénario de test, sont présentées au chapitre 3. Concevoir ce mécanisme sans le tester serait toutefois insuffisant au regard de l'exigence EF-07 : trois scénarios de comportements anormaux contrôlés sont donc conçus à ce stade, pour implémentation et exécution au chapitre 3. Le tableau 2.9 en résume la logique de conception retenue : outil mobilisé sur le poste Kali Linux, hôte cible, principe de détection Zabbix envisagé et sévérité d'alerte associée. Les trois scénarios ciblent délibérément le même hôte, Ubuntu Desktop, choisi comme représentant d'un poste utilisateur Linux type, afin de concentrer l'analyse comparative des trois mécanismes de détection sur un même hôte de référence plutôt que de disperser l'effort de validation sur plusieurs cibles.

TABLE 2.9 – Conception des scénarios de test de sécurité.

Scénario	**Outil (Kali)**	**Hôte cible**	**Principe de détection Zabbix envisagé**	**Sévérité**
Force brute d'authentification SSH	Hydra [17]	Ubuntu Desktop	Comptage des occurrences Failed password dans le journal d'authentification via un item de type *log*	High
Scan de reconnaissance réseau	Nmap [16]	Ubuntu Desktop	Comptage des paquets bloqués par un pare-feu hôte journalisant les connexions refusées, via un item de type *log*	High
Rafale de trafic réseau (saturation de lien)	hping3 [18]	Ubuntu Desktop	Dépassement de seuil sur le débit entrant de l'interface réseau (item `net.if.in`), réutilisant un élément natif du modèle de supervision agent	High

Cette conception établit la correspondance entre chaque comportement anormal généré depuis Kali Linux et le mécanisme de détection Zabbix mobilisé pour l'identifier – sans recourir à un système de détection d'intrusion (IDS) dédié, absent du périmètre de ce projet, mais en exploitant les capacités natives de collecte par agent et par journal d'événements de Zabbix. Ce choix délibéré de ne pas ajouter d'IDS mérite d'être justifié : un tel système aurait certes élargi la couverture de détection, mais au prix d'un composant supplémentaire à déployer, configurer

et maintenir, contrevenant directement à la contrainte de compétences internes limitées (tableau 2.4) sans que sa valeur ajoutée soit démontrée pour les trois techniques d'attaque retenues – toutes trois productrices d'une trace directement exploitable par les mécanismes déjà natifs de Zabbix, comme l'établit le rattachement au référentiel MITRE ATT&CK opéré au chapitre 1 (tableau 1.1.3).

Le principe de détection du scénario de scan de reconnaissance, initialement envisagé sur la seule base du volume de trafic entrant, a par ailleurs nécessité, lors de sa mise en œuvre effective, un raffinement vers l'analyse des journaux d'un pare-feu hôte local – ce volume s'étant révélé insuffisamment discriminant pour isoler fiablement un scan de type *SYN* de la variabilité normale du trafic. Cette adaptation méthodologique, ainsi que la mise en œuvre technique complète des trois scénarios et la vérification effective de la réception des courriels d'alerte correspondants, sont présentées au chapitre 3.

2.5 Modélisation UML de la solution

2.5.1 Acteurs et diagramme de cas d'utilisation

Trois acteurs suffisent à décrire le fonctionnement du système de supervision (tableau 2.10) : l'administrateur DSI, le serveur Zabbix, et l'hôte supervisé. Le poste Kali Linux n'est volontairement pas modélisé comme quatrième acteur : conformément à sa nature établie en section 2.3, il constitue un **stimulus externe du scénario de test**, à l'origine indirecte des événements détectés par le *Serveur Zabbix*, mais n'interagit à aucun moment avec le système au sens strict du diagramme de cas d'utilisation – une frontière de modélisation qui traduit fidèlement la frontière fonctionnelle déjà établie au tableau 2.8.

TABLE 2.10 – Identification des acteurs du système de supervision

Acteur	Description
Administrateur DSI	Agent de la DSI chargé de configurer les hôtes supervisés, de consulter les tableaux de bord et de traiter les alertes
Serveur Zabbix	Système assurant la collecte, le traitement et la notification automatique des événements de supervision
Hôte supervisé	Poste, serveur ou équipement réseau dont l'état est surveillé

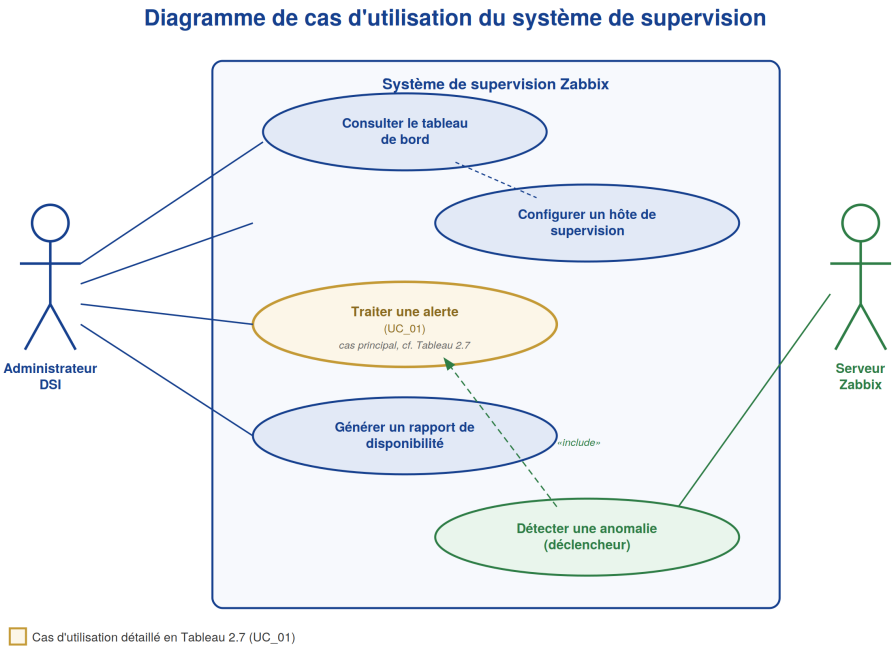


FIGURE 2.3 – Diagramme de cas d'utilisation du système de supervision

Le tableau 2.11 présente la description textuelle du cas d'utilisation principal, *Traiter une alerte*, retenu comme cas nominal car il concentre l'ensemble de la valeur ajoutée du projet : c'est ce scénario, et lui seul, que les trois tests de sécurité du chapitre 3 ont pour vocation de déclencher et de vérifier de bout en bout.

TABLE 2.11 – Description textuelle du cas d'utilisation UC_01 – Traiter une alerte

Identifiant	UC_01
Acteur principal	Administrateur DSI
Précondition	Un hôte supervisé est déclaré dans Zabbix, avec au moins un déclencheur configuré
Scénario nominal	1) Un déclencheur passe à l'état *Problem* ; 2) le serveur génère un événement et déclenche la notification ; 3) l'administrateur reçoit un courriel ; 4) il consulte le tableau de bord ; 5) il intervient sur l'hôte concerné ; 6) le déclencheur repasse à *Resolved*.
Postcondition	L'incident est résolu et historisé

2.5.2 Diagramme de séquence : cycle de détection et de notification

La figure 2.4 détaille temporellement le scénario nominal du tableau 2.11, en explicitant les échanges entre l'hôte supervisé, le serveur Zabbix et l'administrateur DSI.

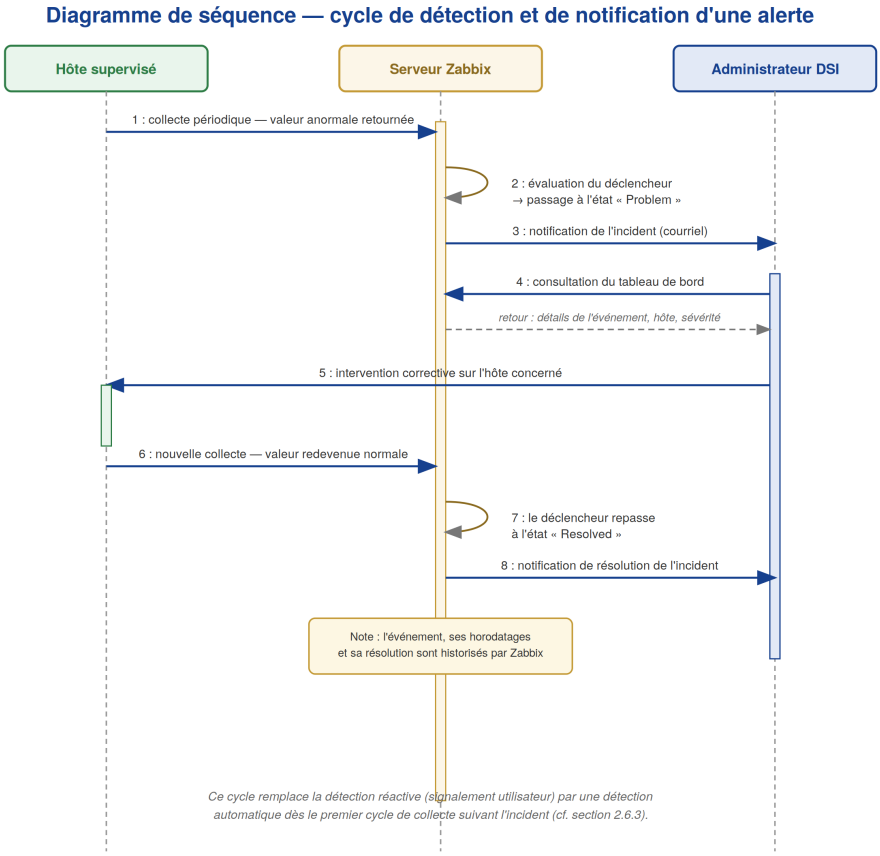


FIGURE 2.4 – Diagramme de séquence du cycle de détection d'une alerte

Ce cycle, exécuté au chapitre 3 pour chacun des trois scénarios du tableau 2.9, illustre la valeur ajoutée centrale de la solution proposée : la détection n'est plus subordonnée à un signalement utilisateur, mais déclenchée automatiquement dès le premier cycle de collecte suivant le comportement anormal généré depuis le poste Kali Linux. C'est précisément l'écart entre ce cycle théorique et sa vérification effective par des mesures de délai, présentée au chapitre 3, qui distingue une architecture correctement *conçue* d'une architecture dont le fonctionnement a été *démontré*.

2.6 Démarche de transposition vers la production

L'ensemble de l'architecture décrite dans ce chapitre a été conçu pour une transposition fidèle vers l'infrastructure réelle de la CDEC, à l'exception explicite du poste Kali Linux. La démarche de transposition prévue repose sur les principes suivants :

- **réutilisation à l'identique** des configurations Zabbix validées en simulation (groupes d'hôtes, modèles, déclencheurs, actions de notification), après vérification préalable de la

cohérence des sévérités configurées et des fuseaux horaires du serveur, seules les adresses IP devant ensuite être adaptées au plan réel de la CDEC ;

- **exclusion du poste de test** : Kali Linux et les scénarios d'attaque associés ne sont en aucun cas reproduits sur l'infrastructure de production ; ils ont exclusivement vocation à valider, en amont, que le mécanisme d'alerte fonctionne correctement avant sa mise en service réelle ;
- **activation progressive de la supervision**, en commençant par les équipements les moins critiques avant les équipements réseau de production ;
- **montée en version de sécurité SNMP**, avec migration de SNMPv2c vers SNMPv3 pour l'environnement de production, conformément à la recommandation du chapitre 1 ;
- **réétalonnage des seuils de déclenchement** sur la ligne de base réelle du trafic et des journaux de la CDEC, les seuils retenus en laboratoire ayant été calibrés sur un environnement à très faible charge.

Bilan du chapitre

Ce chapitre a présenté la démarche méthodologique adoptée, l'analyse de l'existant, la spécification des exigences et l'architecture globale retenue – intégrant, au-delà des trois hôtes supervisés, un poste de test de sécurité sous Kali Linux destiné à valider expérimentalement, et non uniquement en théorie, le mécanisme d'alerte de la solution. La conception des trois scénarios de test associés (force brute SSH, scan de reconnaissance, rafale réseau), tous ciblant l'hôte Ubuntu Desktop, établit le pont direct vers le chapitre 3, qui en présente la mise en œuvre technique complète – y compris l'adaptation du mécanisme de détection du scénario de scan –, les résultats obtenus et l'analyse critique de la démarche de transposition vers la production.

CHAPITRE 3

IMPLÉMENTATION ET RÉSULTATS

Ce chapitre présente la mise en œuvre technique complète de l'architecture conçue au chapitre 2 : environnement de virtualisation, équipements réseau simulés, serveur de supervision Zabbix, hôtes supervisés, puis configuration effective du mécanisme d'alerte par courriel. La seconde partie détaille le déploiement du poste de test Kali Linux et l'exécution des trois scénarios de comportements anormaux contrôlés, avec les commandes exactes utilisées, les mécanismes de détection configurés et les résultats mesurés, avant une analyse critique et la démarche de transposition vers la production.

Contents

3.1 Mise en place de l'environnement de virtualisation **30**
3.2 Configuration des équipements réseau **32**
3.3 Installation et configuration du serveur Zabbix **33**
3.4 Déploiement de la supervision des hôtes **34**
3.5 Configuration du mécanisme d'alerte **37**
3.6 Exécution des scénarios de sécurité **38**
3.6.1 Scénario 1 : force brute d'authentification SSH (Hydra) 39
3.6.2 Scénario 2 : scan de reconnaissance réseau (Nmap) 40
3.6.3 Scénario 3 : rafale de trafic réseau (hping3) 41
3.6.4 Synthèse des résultats et mesure du délai de détection 41
3.7 Analyse critique des résultats et recommandations **43**
3.8 Démarche de transposition vers la production **44**
Bilan du chapitre **44**

3.1 Mise en place de l'environnement de virtualisation

Réseaux virtuels et intégration GNS3/VMware. L'intégration de GNS3 avec des machines virtuelles VMware indépendantes repose sur la création d'un réseau virtuel dédié de type *Host-only* (`VMnet2`), via le *Virtual Network Editor* de VMware Workstation [11], configuré sans service DHCP et sans passerelle vers le réseau physique de la machine hôte, garantissant l'isolement complet du laboratoire. Le pont entre la topologie GNS3 et ces machines s'effectue par l'ajout d'un nœud de type *Cloud* lié à cette interface [12], relié au commutateur `Switch2` de la topologie du chapitre 2 (figure 2.2.2), établissant la continuité de niveau 2 entre le pare-feu pfSense simulé et les machines virtuelles hébergeant le serveur Zabbix, les hôtes supervisés et le poste de test Kali Linux.

La figure 3.1 présente la topologie ainsi câblée et opérationnelle au sein du canevas GNS3, l'ensemble des liens apparaissant en vert (statut actif). Elle permet d'observer concrètement deux aspects abordés jusqu'ici de façon uniquement théorique : la répartition des méthodes d'accès aux équipements réseau simulés, et la nature hybride du moteur d'exécution sous-jacent à GNS3.

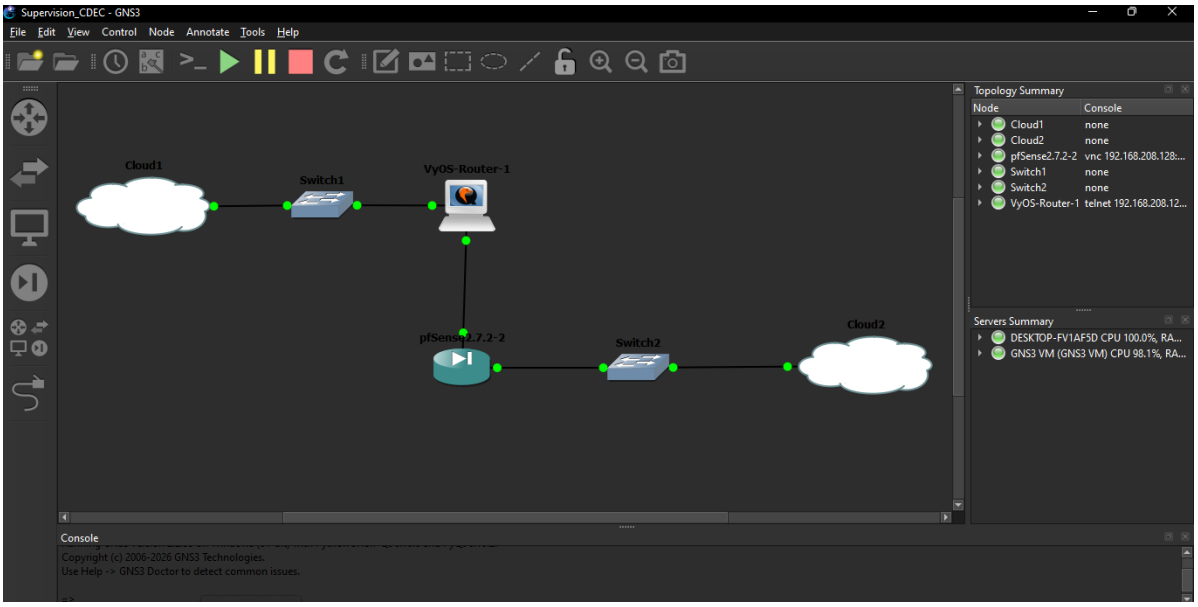


FIGURE 3.1 – Topologie de supervision de la CDEC opérationnelle dans GNS3

Cette capture apporte trois éléments de lecture complémentaires au principe d'architecture exposé plus haut.

Premièrement, le panneau *Topology Summary* révèle que les deux équipements réseau simulés ne sont pas administrés de la même façon depuis GNS3 : le routeur `VyOS-Router-1` est accessible par une console **Telnet** (`192.168.128.12...`), conforme à son mode d'administration en ligne de commande, tandis que `pfSense2.7.2-2` est accessible par une console **VNC** (`192.168.128.128...`), cohérent avec son interface de gestion graphique/web décrite

au chapitre 1. Cette différence illustre concrètement, au niveau de l'outil de simulation lui-même, la complémentarité des deux équipements déjà justifiée au chapitre 2 (tableau 2.2.6).

Deuxièmement, le panneau *Servers Summary* met en évidence la nature hybride du moteur d'exécution : GNS3 répartit la charge de simulation entre deux serveurs distincts, le poste hôte local (`DESKTOP-FV1AF5D`) et une machine virtuelle dédiée (`GNS3 VM`), cette dernière hébergeant l'essentiel des nœuds réseau (VyOS, pfSense, commutateurs) afin d'isoler leur exécution de celle du système d'exploitation hôte. C'est cette GNS3 VM, et non le poste hôte directement, qui communique avec les machines virtuelles VMware Workstation via les nœuds `Cloud1` et `Cloud2`.

Troisièmement, la chaîne `Cloud1` → `Switch1` → `VyOS-Router-1` → `pfSense2.7.2-2` → `Switch2` → `Cloud2` matérialise exactement le sens de traversée du trafic défini au chapitre 2 : `Cloud1` représente le point d'entrée externe (WAN), tandis que `Cloud2` constitue le pont de sortie vers le réseau interne `VMnet2` de VMware Workstation, sur lequel sont effectivement connectées les machines virtuelles supervisées – serveur Zabbix, Ubuntu Desktop, serveur Windows et poste Kali Linux. Aucune de ces quatre machines n'apparaît comme nœud GNS3 dans ce canevas : conformément au principe énoncé en section 3.1, elles restent des machines virtuelles VMware indépendantes, simplement rendues joignables par ce pont réseau, et non des composants internes à la simulation GNS3 elle-même.

Provisionnement des machines virtuelles. Le tableau 3.1 résume les caractéristiques techniques retenues pour chacune des machines virtuelles du laboratoire, ainsi que l'adresse IP effectivement attribuée conformément au plan d'adressage du chapitre 2.

TABLE 3.1 – Provisionnement des machines virtuelles du laboratoire

Machine	Système	vCPU	RAM	Disque	Rôle / Adresse IP
Serveur Zabbix	Ubuntu Server 24.04 LTS	2	4 Go	40 Go	Serveur de supervision – 192.168.47.50
Poste supervisé 1	Ubuntu Desktop 24.04 LTS	2	4 Go	30 Go	Hôte supervisé (agent) – 192.168.47.30
Poste supervisé 2	Windows Server	2	4 Go	40 Go	Hôte supervisé (agent) – 192.168.47.20
Poste de test	Kali Linux 2026.x	2	2 Go	20 Go	Génération de comportements anormaux – 192.168.47.66

Ce tableau confirme que le dimensionnement retenu reste volontairement modeste (2 vCPU, 2 à 4 Go de RAM par machine), cohérent avec la contrainte de fonctionnement sur une machine hôte unique en virtualisation imbriquée identifiée au chapitre 2.

3.2 Configuration des équipements réseau

Routeur VyOS. La configuration de l'interface interne du routeur VyOS et l'activation du service SNMP nécessaire à sa supervision s'effectuent en mode configuration [13] :

VyOS – interface et SNMP

```
configure
set interfaces ethernet eth1 address '10.0.0.1/30'
set service snmp community public authorization ro
set service snmp listen-address 10.0.0.1 port 161
set service snmp listen-address 127.0.0.1 port 161
commit
save
```

Conformément à l'observation du chapitre 1 (section 1.2), VyOS restreint par défaut l'écoute SNMP à l'interface de bouclage : la double déclaration de `listen-address` ci-dessus est indispensable pour que le serveur Zabbix, distant, puisse effectivement interroger l'agent SNMP du routeur.

Pare-feu pfSense. La configuration de pfSense s'effectue via son interface web [14] : attribution de `10.0.0.2/30` à l'interface WAN et de `192.168.47.254/24` à l'interface LAN, activation du service SNMP (*Services* > *SNMP*, communauté `public` en lecture seule), et création d'une règle de pare-feu autorisant le trafic SNMP (UDP/161) entrant depuis l'adresse du serveur Zabbix uniquement.

Vérification de la connectivité de bout en bout. Avant toute installation logicielle, la connectivité entre l'ensemble des nœuds a été vérifiée par une série de tests `ping`, synthétisés au tableau 3.2.

TABLE 3.2 – Résultats des tests de connectivité inter-hôtes

Source	Destination	Résultat
Routeur VyOS (eth1)	Pare-feu pfSense (WAN)	✓
Pare-feu pfSense (LAN)	Serveur Zabbix	✓
Serveur Zabbix	Ubuntu Desktop	✓
Serveur Zabbix	Serveur Windows	✓
Kali Linux (192.168.47.66)	Ubuntu Desktop (passerelle 192.168.47.254)	✓

Ce tableau valide la continuité IP de bout en bout de la topologie avant toute couche applicative : aucune installation de service n'a été entreprise tant que l'ensemble des cinq liaisons n'était pas confirmé.

3.3 Installation et configuration du serveur Zabbix

Pile logicielle. Le serveur de supervision repose sur Ubuntu Server 24.04 LTS. Après mise à jour du système, la pile Apache/MySQL puis Zabbix Server 7.0 LTS sont installées selon la procédure officielle du dépôt Zabbix pour Ubuntu 24.04 [6] :

Installation de la pile Apache / MySQL / Zabbix Server

```
sudo apt update && sudo apt upgrade -y
sudo apt install apache2 mysql-server -y
sudo mysql_secure_installation

wget https://repo.zabbix.com/zabbix/7.0/ubuntu/pool/main/z/zabbix-
   release/zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo dpkg -i zabbix-release_latest_7.0+ubuntu24.04_all.deb
sudo apt update
sudo apt install zabbix-server-mysql zabbix-frontend-php zabbix-apache-
   conf zabbix-sql-scripts zabbix-agent2 -y
```

La base de données dédiée est ensuite créée et initialisée avant démarrage des services :

Initialisation de la base de données et démarrage des services

```
mysql -uroot -p
create database zabbix character set utf8mb4 collate utf8mb4_bin;
create user zabbix@localhost identified by 'MotDePasseFort';
grant all privileges on zabbix.* to zabbix@localhost;
set global log_bin_trust_function_creators = 1;
quit;

zcat /usr/share/zabbix-sql-scripts/mysql/server.sql.gz | mysql --
   default-character-set=utf8mb4 -uzabbix -p zabbix

sudo systemctl restart zabbix-server zabbix-agent2 apache2
sudo systemctl enable zabbix-server zabbix-agent2 apache2
```

Configuration initiale via l'interface web. L'assistant de configuration web (`http://192.168.47.50/zabbix`) finalise l'installation : vérification des prérequis PHP, connexion à la base de données et création du compte d'administration. La figure 3.2 illustre l'écran d'accueil de cet assistant, première étape visible de la mise en service du serveur.

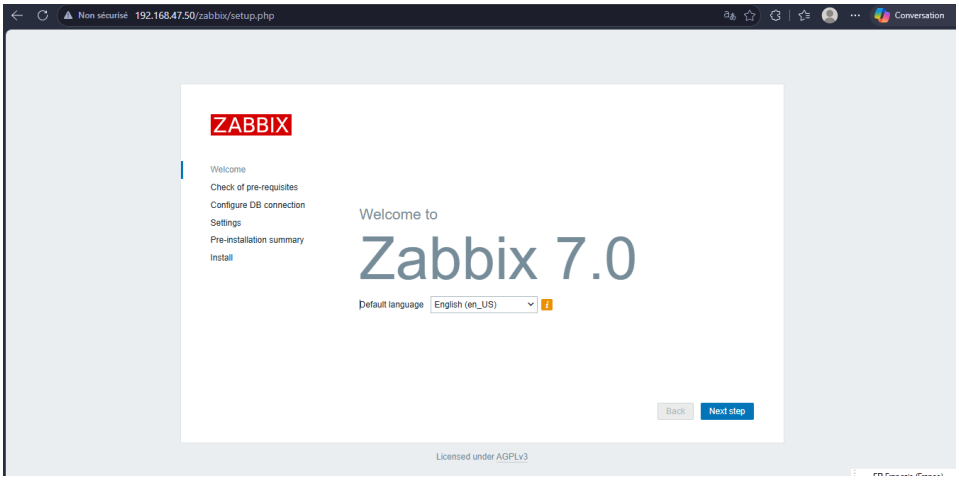


FIGURE 3.2 – Écran d'accueil de l'assistant d'installation de Zabbix 7.0

Cette étape confirme la version effectivement installée (Zabbix 7.0) et marque le point de départ de toute la configuration ultérieure. L'interface a ensuite été basculée en langue française pour l'ensemble de l'exploitation, conformément à la contrainte ENF-05 du chapitre 2.

3.4 Déploiement de la supervision des hôtes

Installation des agents Zabbix. Sur Ubuntu Desktop, l'agent est installé et configuré en mode *actif* :

Configuration de l'agent Zabbix sur Ubuntu Desktop

```
sudo apt install zabbix-agent2 -y
sudo sed -i 's/^Server=.*/Server=192.168.47.50/' /etc/zabbix/
    zabbix_agent2.conf
sudo sed -i 's/^ServerActive=.*/ServerActive=192.168.47.50/' /etc/
    zabbix/zabbix_agent2.conf
sudo sed -i 's/^Hostname=.*/Hostname=Ubuntu-Desktop-CDEC/' /etc/zabbix/
    zabbix_agent2.conf
sudo systemctl restart zabbix-agent2
```

Ce mode actif exige une correspondance stricte entre le paramètre `Hostname` du fichier local et le nom de l'hôte déclaré côté serveur : tout écart, y compris de casse, empêche silencieusement la remontée des éléments actifs, notamment les éléments de journal utilisés en section 3.6.

Sur le serveur Windows, l'agent MSI officiel est installé via son assistant graphique (figure 3.3), avec les mêmes paramètres de serveur et `Hostname=WinServer-CDEC`.

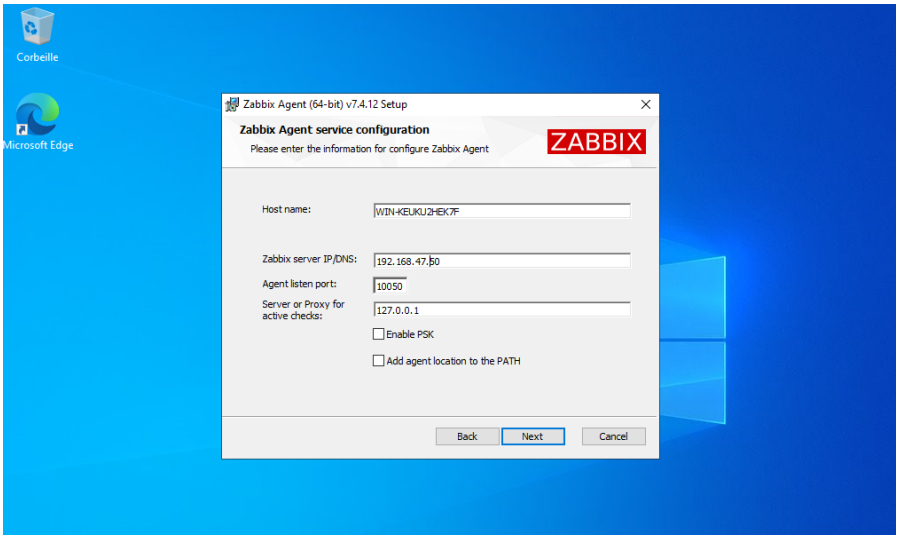


FIGURE 3.3 – Assistant d'installation de l'agent Zabbix sur Windows Server.

Cette capture illustre la simplicité de déploiement de l'agent sur un environnement Windows fermé à l'administration en ligne de commande, argument déterminant retenu au chapitre 1 en faveur de la supervision par agent pour les systèmes ouverts.

Déclaration des hôtes et supervision SNMP. Aucun modèle officiel n'étant fourni par défaut pour VyOS et pfSense, deux éléments SNMP génériques ont été créés manuellement pour chacun : `sysUpTime` (`1.3.6.1.2.1.1.3.0`) pour la disponibilité, et `sysDescr` (`1.3.6.1.2.1.1.1.0`) pour l'identification, comme le montre la figure 3.4 pour le routeur VyOS. Un déclencheur associé à `sysUpTime` avec la fonction `nodata()` détecte l'absence de réponse de l'équipement, révélatrice d'une indisponibilité.

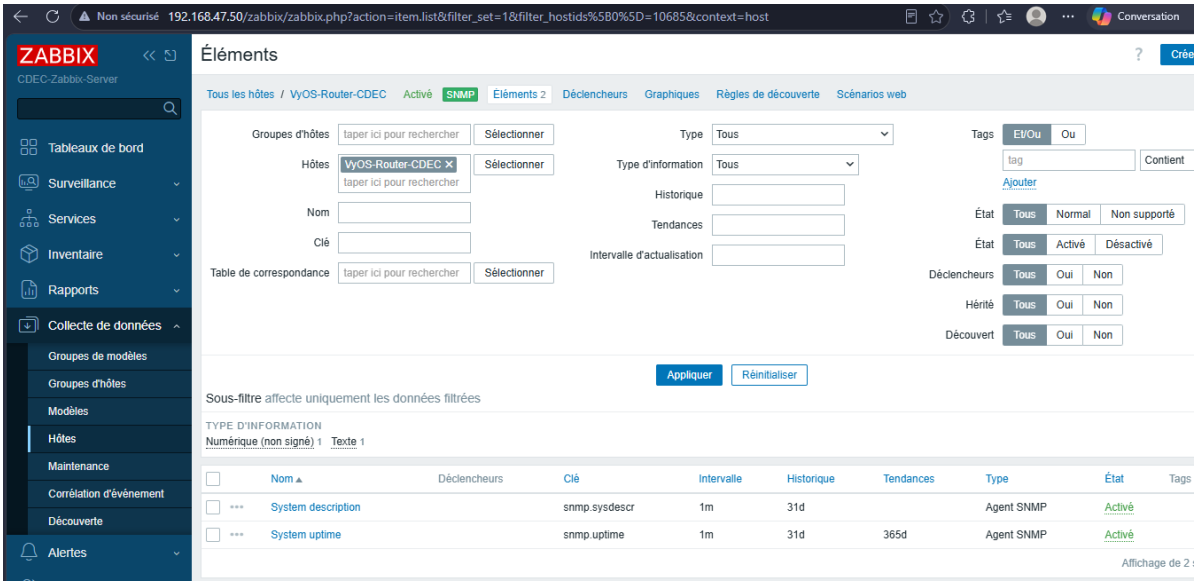


FIGURE 3.4 – Éléments SNMP configurés pour le routeur VyOS-Router-CDEC

Le tableau 3.3 et la figure 3.5 confirment la déclaration effective des cinq hôtes du périmètre, chacun rattaché au modèle de supervision adapté à sa nature.

TABLE 3.3 – Déclaration des hôtes supervisés dans Zabbix

Nom de l'hôte	Groupe	Adresse	Modèle appliqué
Zabbix server	CDEC – Serveurs	127.0.0.1 (agent local)	Linux by Zabbix agent, Zabbix server health
Ubuntu-Desktop-CDEC	CDEC – Postes	192.168.47.30	Linux by Zabbix agent
WinServer-CDEC	CDEC – Serveurs	192.168.47.51	Windows by Zabbix agent
VyOS-Router-CDEC	CDEC – Réseau	10.0.0.1	Modèle SNMP personnalisé
pfSense-Firewall-CDEC	CDEC – Réseau	192.168.47.254	Modèle SNMP personnalisé

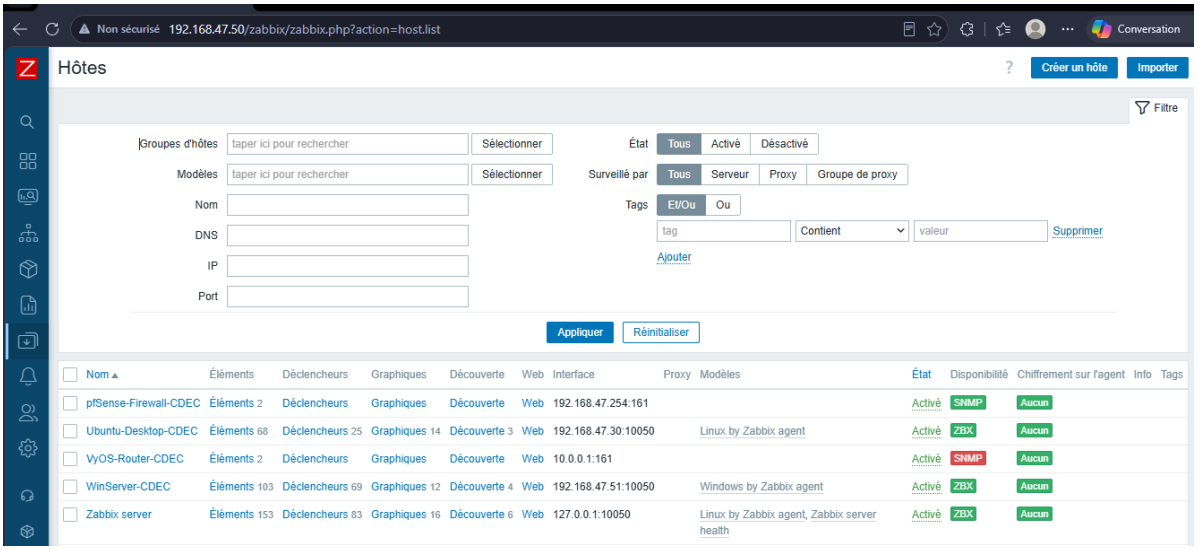


FIGURE 3.5 – Liste des hôtes déclarés dans le serveur Zabbix de la CDEC

Cette capture confirme la cohérence entre la conception du chapitre 2 et l'implémentation réalisée : cinq hôtes sont déclarés et actifs, tous en disponibilité *ZBX* ou *SNMP* selon leur nature, et le poste Kali Linux – volontairement absent de cette liste – confirme le choix de ne pas le traiter comme un actif supervisé.

Vérification de la collecte des données. La consultation de *Surveillance* > *Dernières données* confirme la remontée effective des métriques pour l'ensemble des hôtes déclarés (70 éléments actifs pour le seul hôte Ubuntu-Desktop-CDEC), avec un intervalle de rafraîchissement de l'ordre de la minute pour les métriques système et de 10 secondes pour les éléments de journal configurés pour les scénarios de sécurité (section 3.6).

3.5 Configuration du mécanisme d'alerte

Type de média et compte d'envoi. Le canal de notification retenu est le courrier électronique, envoyé via le serveur SMTP de Google (tableau 3.4).

TABLE 3.4 – Paramètres de configuration du type de média Email

Paramètre	Valeur retenue
Serveur SMTP	smtp.gmail.com
Port SMTP	587
Sécurité de la connexion	STARTTLS
Authentification	Nom d'utilisateur et mot de passe (mot de passe d'application)
Courriel expéditeur	minkande62@gmail.com

L'authentification SMTP standard n'étant pas acceptée par Google pour les applications tierces, un **mot de passe d'application** dédié a été généré depuis les paramètres de sécurité du compte (validation en deux étapes activée), plutôt que le mot de passe principal du compte. La fonctionnalité de test intégrée à Zabbix (*Types de médias* > *Email* > *Test*) a permis de valider l'authentification SMTP avant toute mise en service.

Média utilisateur et action de notification. Un média de type *Email* est associé à l'utilisateur administrateur, avec un filtre de sévérité minimale fixé à *Moyen*, plus permissif que la seule sévérité *Haut* initialement envisagée, afin de couvrir l'ensemble des scénarios de la section 3.6. L'action reliant effectivement un déclencheur à l'envoi du courriel est résumée au tableau 3.5 et illustrée à la figure 3.6.

TABLE 3.5 – Configuration de l'action de notification des incidents de sécurité

Paramètre	Valeur retenue
Nom de l'action	Alerte sécurité – Notification mail
Condition	Sévérité du déclencheur ≥ Moyen
Opération	Envoyer un message à l'utilisateur Admin, tous médias disponibles
Étapes / Durée	1–1 / 0 (envoi unique, sans escalade)

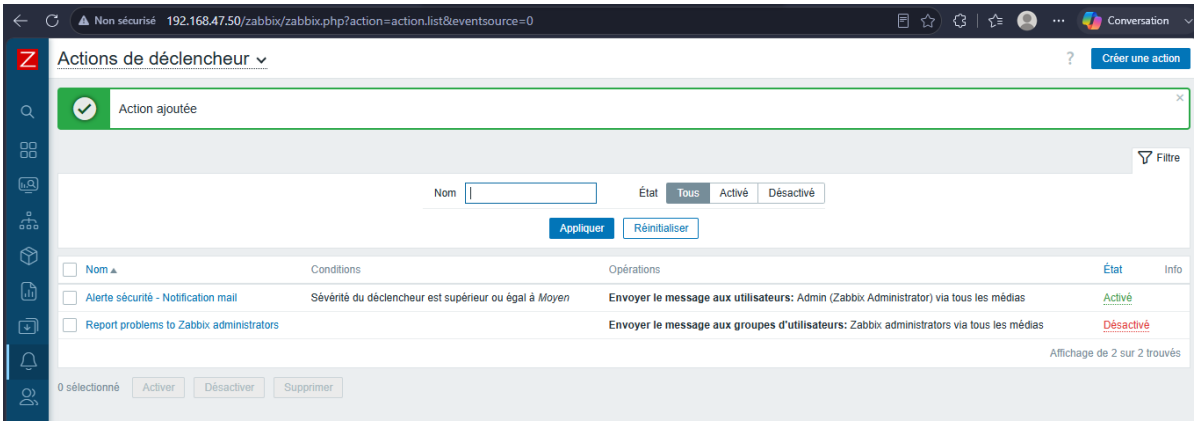


FIGURE 3.6 – Confirmation de la création de l'action de notification.

Cette action, et elle seule, relie effectivement le passage d'un déclencheur à l'état *Problem* à l'envoi réel d'un courriel : sa validation explicite, avant toute exécution des scénarios de sécurité, s'est révélée nécessaire pour clore la chaîne complète collecte → déclencheur → notification.

3.6 Exécution des scénarios de sécurité

Une machine virtuelle Kali Linux est provisionnée sur le réseau interne (`192.168.47.66/24`), sans déclaration comme hôte dans Zabbix, conformément à la

conception du chapitre 2. Pour chacun des trois scénarios suivants, la même démarche en quatre temps est appliquée : identification d'une source de journalisation ou d'une métrique révélatrice du comportement recherché ; création d'un élément Zabbix de type *journal* ou réutilisation d'un élément natif de l'agent ; création d'un déclencheur avec expression de résolution (*Expression OK*) ; génération du comportement depuis Kali Linux et vérification de la réception du courriel. Le tableau 3.6 synthétise la configuration technique retenue pour les trois scénarios avant le détail de leur exécution.

TABLE 3.6 – Synthèse de la configuration de détection des trois scénarios de sécurité

Scénario	Source / clé Zabbix	Condition du déclencheur	Sévérité
Force brute SSH	`log[/var/log/auth.log, "Failed password"]`	`count()` ≥ 5 occurrences sur 5 min	Haut
Scan de ports	`log[/var/log/syslog, "UFW BLOCK"]`	`count()` ≥ 10 occurrences sur 1 min	Haut
Rafale réseau	`net.if.in["ens33"]` (élément natif)	`avg()` > 1 Mbps sur 1 min	Haut

3.6.1 Scénario 1 : force brute d'authentification SSH (Hydra)

Le service SSH est activé sur Ubuntu Desktop, et l'utilisateur `zabbix` de l'agent est ajouté au groupe `adm` pour lire le journal d'authentification :

Préparation de la cible

```
sudo systemctl enable --now ssh
sudo usermod -aG adm zabbix
sudo systemctl restart zabbix-agent2
```

Un élément Zabbix de type journal, actif, collecte chaque occurrence de la chaîne Failed password dans `/var/log/auth.log`, avec un intervalle de mise à jour de 10 secondes. Depuis Kali Linux, une attaque par dictionnaire est ensuite menée contre le service SSH de l'hôte cible [17] :

Attaque par dictionnaire depuis Kali

```
sudo gunzip /usr/share/wordlists/rockyou.txt.gz
hydra -l minkande -P /usr/share/wordlists/rockyou.txt ssh
   ://192.168.47.30 -t 4
```

En quelques secondes, cinq tentatives échouées ou plus sont enregistrées, franchissant le seuil du déclencheur, qui passe à l'état *Problem* et déclenche la notification (figure 3.8, présentée

en synthèse en fin de section).

3.6.2 Scénario 2 : scan de reconnaissance réseau (Nmap)

La conception du chapitre 2 envisageait de détecter le scan par une élévation du volume de trafic entrant. En pratique, un scan de type *SYN* ne génère pas un volume suffisamment distinct de la ligne de base pour être fiablement isolé par ce seul indicateur. La détection a donc été réalisée par activation du pare-feu local **UFW** en mode journalisation sur l'hôte cible :

Activation du pare-feu local UFW avec journalisation

```
sudo ufw allow ssh
sudo ufw allow 10050/tcp
sudo ufw logging on
sudo ufw --force enable
```

Les règles autorisant explicitement SSH (22) et l'agent Zabbix (10050) garantissent que l'activation du pare-feu local ne compromet pas la supervision de l'hôte lui-même. Sur les versions récentes d'Ubuntu, les événements `UFW BLOCK` sont journalisés par le sous-système `kernel` dans `/var/log/syslog` plutôt que dans un fichier `ufw.log` dédié, désormais absent par défaut – l'élément Zabbix a été configuré en conséquence (tableau 3.6). Depuis Kali Linux, un scan par paquets *SYN* est exécuté contre les 1000 ports les plus courants de l'hôte cible [16] :

Scan de reconnaissance depuis Kali

```
nmap -sS -p 1-1000 192.168.47.30
```

Chaque paquet atteignant un port fermé ou filtré est bloqué et journalisé par UFW, générant en quelques secondes bien plus des dix occurrences requises sur la fenêtre d'une minute. Le courriel reçu suite à ce scénario, illustré à la figure 3.7, confirme la remontée de la donnée opérationnelle brute issue du journal UFW jusqu'à l'administrateur.

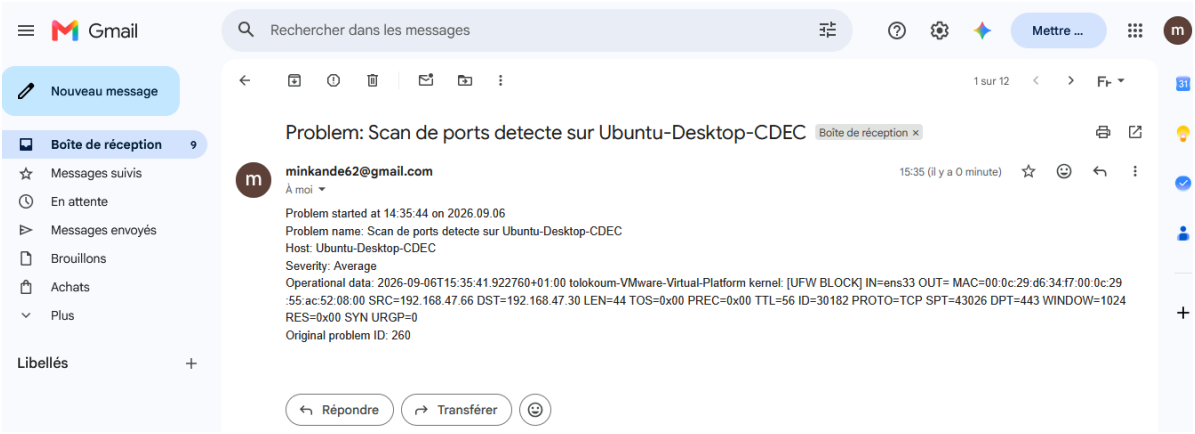


FIGURE 3.7 – Courriel de notification reçu suite au déclenchement du scénario de test

Cette capture met en évidence un écart à corriger avant la production : la sévérité affichée (*Average*) diffère de la sévérité *Haut* initialement configurée pour ce déclencheur – point discuté en section 3.7.

3.6.3 Scénario 3 : rafale de trafic réseau (hping3)

Ce troisième scénario ne nécessite la création d'aucun nouvel élément : il s'appuie directement sur l'élément natif `Interface ens33: Bits received` (`net.if.in["ens33"]`), déjà collecté par le modèle *Linux by Zabbix agent*. Le trafic entrant observé au repos se situant entre 3 et 6 Kbps, un seuil de 1 Mbps – environ 200 fois la ligne de base – a été retenu afin d'écarter tout risque de faux positif. Depuis Kali Linux, une rafale de paquets *SYN* est générée en continu contre le port 80 de l'hôte cible [18] :

Rafale de trafic depuis Kali

```
sudo hping3 -S --flood -p 80 192.168.47.30
```

La rafale est maintenue une à deux minutes, temps nécessaire pour que la moyenne glissante d'une minute dépasse le seuil configuré, avant interruption manuelle (`Ctrl+C`).

3.6.4 Synthèse des résultats et mesure du délai de détection

Le tableau 3.7 synthétise, pour chacun des trois scénarios, la commande exécutée, le mécanisme de détection effectivement mis en œuvre, la sévérité observée et le résultat de la notification.

TABLE 3.7 – Synthèse comparative des trois scénarios de test de sécurité

Scénario	Commande (Kali)	Sévérité observée	Courriel reçu
Force brute SSH (Hydra)	`hydra -l minkande -P rockyou.txt ssh://192.168.47.30`	Haut	✓
Scan de ports (Nmap)	`nmap -sS -p 1-1000 192.168.47.30`	Moyen[1]	✓
Rafale réseau (hping3)	`hping3 -S --flood -p 80 192.168.47.30`	Haut	✓

Les trois scénarios ont chacun donné lieu à la réception effective d'un courriel d'alerte, confirmant le fonctionnement de bout en bout de la chaîne collecte → déclencheur → action → notification. La figure 3.8 présente le tableau de bord global consultable par l'administrateur DSI, montrant simultanément plusieurs incidents actifs classés par sévérité.

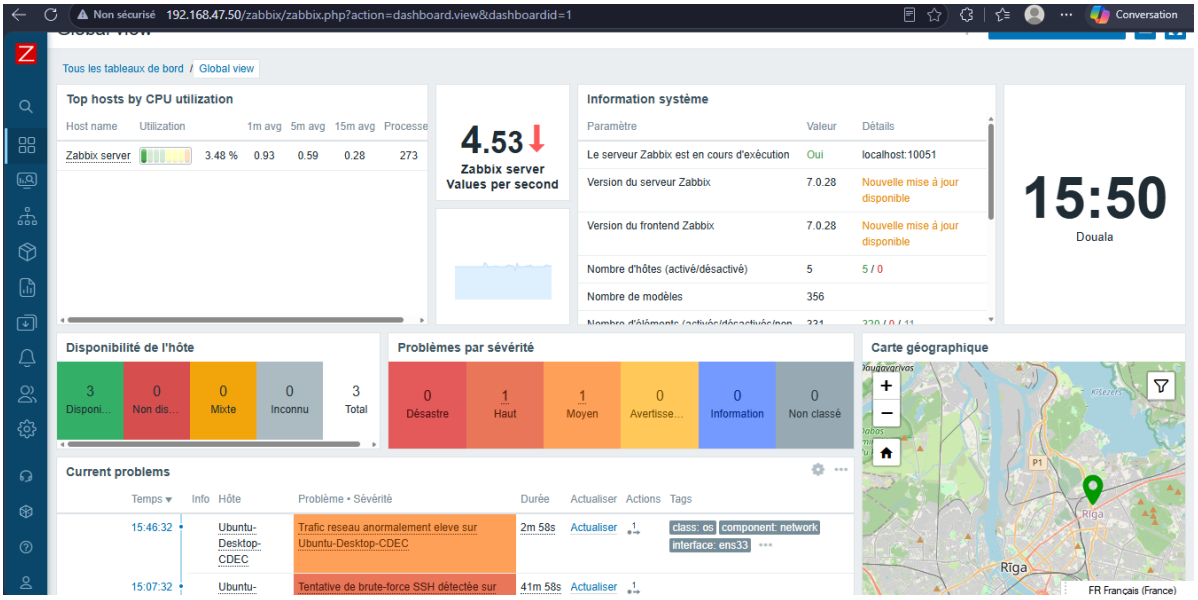


FIGURE 3.8 – Tableau de bord Zabbix présentant plusieurs incidents de sécurité actifs

Cette figure illustre un apport opérationnel notable de la solution : la capacité du tableau de bord à présenter, en un seul point de consultation, l'ensemble des incidents actifs classés par sévérité – fonctionnalité directement exploitable par la DSI pour prioriser son intervention, conformément à l'exigence EF-05 du chapitre 2.

Pour chaque scénario, l'heure de première occurrence exploitable, l'heure de passage à l'état *Problem* et l'heure de réception du courriel ont été relevées, permettant d'approcher le temps moyen de détection (MTTD) du dispositif (tableau 3.8).

1. Sévérité observée en exploitation (Moyen), bien que le déclencheur ait été paramétré en sévérité Haut lors de sa création – écart discuté en section 3.7.

TABLE 3.8 – Estimation du temps moyen de détection et de notification par scénario.

Scénario	Intervalle de collecte	Condition de déclenchement	Délai jusqu'à *Problem*	Délai jusqu'à courriel
Force brute SSH	10 s	5e tentative échouée (fenêtre 5 min)	~10–15 s	< 1 min
Scan de ports	10 s	10e paquet bloqué (fenêtre 1 min)	~10–15 s	< 1 min
Rafale réseau	60 s (natif)	Débit moyen > 1 Mbps sur 1 min	≤ 60 s	< 1 min

Ces mesures, bien que réalisées à l'échelle d'un laboratoire à faible charge, confirment un ordre de grandeur de détection et de notification systématiquement inférieur à deux minutes, à comparer à la situation initiale du chapitre 2, où la détection dépendait exclusivement d'un signalement utilisateur, potentiellement différé de plusieurs heures.

3.7 Analyse critique des résultats et recommandations

Performance de détection. Les trois scénarios confirment que la chaîne complète de détection fonctionne de bout en bout, conformément au diagramme de séquence conçu au chapitre 2 (figure 2.2.4). En reprenant les indicateurs du chapitre 1 (équations 1.2 à 1.4), le délai observé entre l'action malveillante et la réception du courriel constitue une approximation du MTTR minimal atteignable, très largement inférieur au délai de détection par signalement utilisateur caractérisant la situation initiale.

Limites identifiées.

- **Écart de sévérité observé sur le scénario de scan** : le déclencheur a été configuré en sévérité *Haut*, mais les événements observés en exploitation sont apparus en sévérité *Moyen*. Cet écart, non résolu au moment de la rédaction, doit être vérifié en priorité avant tout déploiement en production ;
- **Décalage horaire** d'environ une heure constaté entre l'horodatage affiché dans le corps du courriel et celui du tableau de bord, vraisemblablement imputable à une divergence entre le fuseau horaire du serveur (UTC) et celui d'affichage utilisateur (Africa/Douala, UTC+1) ;
- **Adaptation du mécanisme de détection du scan** par rapport à la conception initiale : le recours à la journalisation du pare-feu local UFW, bien que plus fiable que le seul volume de trafic, introduit une dépendance à un composant de sécurité non présent par défaut sur l'ensemble du parc de la DSI ;
- **Absence de système de détection d'intrusion (IDS) dédié** : la détection reste limitée aux comportements se traduisant par une variation mesurable des journaux ou métriques supervisés ; une attaque plus discrète pourrait ne pas franchir les seuils configurés ;

- **Seuils calibrés empiriquement** sur la ligne de base de ce laboratoire, à très faible charge réelle ; leur transposition en production nécessitera un réétalonnage sur le trafic réel de la CDEC ;
- **SNMPv2c**, utilisé en simulation, ne chiffre pas les échanges ; la migration vers SNMPv3 reste une recommandation ferme avant tout déploiement réel.

Recommandations pour la production. Il est recommandé à la DSI de la CDEC de : (1) vérifier et corriger la sévérité effectivement enregistrée pour chaque déclencheur ; (2) harmoniser le fuseau horaire du serveur et de l'interface Zabbix sur *Africa/Douala* ; (3) conserver les modèles et déclencheurs validés en simulation comme base de départ, en réétalonnant les seuils sur le trafic réel pendant une période probatoire d'au moins deux semaines ; (4) activer SNMPv3 dès le déploiement en production ; (5) dupliquer, à des fins de non-régression uniquement, la démarche de validation par simulation d'incident avant toute modification substantielle des déclencheurs – sans répliquer physiquement le poste Kali Linux hors du laboratoire, conformément à l'exigence ENF-06 du chapitre 2.

3.8 Démarche de transposition vers la production

À la date de rédaction de ce mémoire, l'ensemble de l'architecture décrite dans ce chapitre – y compris le mécanisme d'alerte par courriel et sa validation par les trois scénarios de sécurité – a été intégralement validé en environnement de simulation. Le déploiement sur l'infrastructure réelle de la CDEC constitue la suite du projet, structurée selon la démarche progressive du chapitre 2 (section 2.8) : réutilisation des configurations Zabbix après correction des écarts identifiés en section 3.7, adaptation du plan d'adressage, activation progressive des hôtes en commençant par les moins critiques, et montée en version de sécurité SNMP. Un guide d'exploitation détaillé, destiné à l'équipe DSI, est fourni en annexe.

Bilan du chapitre

Ce chapitre a présenté la mise en œuvre technique intégrale de la solution conçue au chapitre 2 : environnement de virtualisation, équipements réseau simulés, serveur Zabbix, hôtes supervisés et mécanisme d'alerte par courriel, configuré et testé de bout en bout via la fonctionnalité native de test SMTP de Zabbix. L'exécution de trois scénarios de comportements anormaux contrôlés – force brute SSH, scan de ports et rafale de trafic réseau –, générés depuis un poste Kali Linux dédié et non supervisé, a permis de valider expérimentalement, et non uniquement en théorie, que la chaîne complète de détection et de notification fonctionne effectivement, avec un délai systématiquement inférieur à deux minutes. L'analyse critique des résultats, qui met notamment en évidence un écart de sévérité à corriger et une nécessaire harmonisation des fuseaux

horaires, ainsi que la démarche de transposition vers la production qui en découle, concluent ce travail et ouvrent sur la conclusion générale du mémoire.

CONCLUSION GÉNÉRALE

Ce mémoire répond à un besoin identifié au sein de la Direction des Systèmes d'Information de la CDEC : renforcer les mécanismes de supervision existants, encore partiels et non centralisés, par une solution unifiée capable de détecter proactivement les incidents affectant les postes, serveurs et équipements réseau de l'établissement. La démarche conduite en trois temps – état de l'art et positionnement technologique, conception architecturale et de sécurité, implémentation et validation expérimentale – établit deux constats.

— La validation en environnement de simulation (GNS3/VMware Workstation) démontre la viabilité de l'architecture avant toute intervention sur l'infrastructure réelle, conformément au principe valider avant de déployer retenu dès la conception.

— Le mécanisme d'alerte par courriel, mis à l'épreuve par trois scénarios de comportements anormaux contrôlés – force brute SSH, scan de reconnaissance et rafale de trafic –, rattachés au référentiel international MITRE ATT&CK, fonctionne de bout en bout avec un délai de détection et de notification systématiquement inférieur à deux minutes.

Ces résultats s'appuient sur une mesure directe du cycle complet collecte → déclencheur → notification, et sur une analyse critique ayant permis d'identifier des points de vigilance concrets avant tout déploiement.

Trois limites bornent leur généralisation : (i) les seuils de déclenchement, calibrés sur un laboratoire à faible charge, devront être réétalonnés sur le trafic réel de la CDEC ; (ii) la détection reste circonscrite aux techniques produisant une trace exploitable par les mécanismes natifs de Zabbix, en l'absence d'un système de détection d'intrusion dédié ; (iii) SNMPv2c, retenu en simulation à des fins fonctionnelles, ne chiffre pas les échanges et doit céder la place à SNMPv3 en production.

Les perspectives incluent l'activation progressive de la solution sur l'infrastructure réelle selon la démarche définie au chapitre 2, l'élargissement des scénarios de validation à d'autres techniques de la matrice MITRE ATT&CK, et la reconduction périodique de cette démarche de test avant toute évolution substantielle des déclencheurs en production.

En définitive, le renforcement de la supervision d'un établissement financier en structuration ne se limite pas au déploiement d'un outil : sa valeur se mesure à la preuve, et non à la seule promesse, que ses alertes parviendront effectivement à l'administrateur au moment où un incident survient. Les configurations, scripts et résultats de mesure produits dans ce mémoire, directement réutilisables par la DSI, visent à inscrire cette exigence de preuve dans la pratique courante de l'établissement.

RÉFÉRENCES

[1] Kelley DEMPSEY et al. *Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations*. NIST Special Publication 800-137. Consulté le 3 septembre 2026. National Institute of Standards et Technology (NIST), sept. 2011. URL : https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-137.pdf.

[2] RÉPUBLIQUE DU CAMEROUN. *Loi n°2008/003 du 14 avril 2008 régissant les dépôts et consignations*. Rapp. tech. Consulté le 3 septembre 2026. Journal Officiel de la République du Cameroun, avril 2008. URL : https://api.cdec.cm/public/api/documents_reglementation/VBTZAmNwqgy6SKTrOLeCCh579VVVOn35YwFq3w3r.pdf.

[3] RÉPUBLIQUE DU CAMEROUN. *Décret n°2011/105 du 15 avril 2011 portant organisation et fonctionnement de la Caisse des Dépôts et Consignations*. Rapp. tech. Consulté le 3 septembre 2026. Journal Officiel de la République du Cameroun, avril 2011. URL : https://www.juriafrica.com/lex/decret-2011-105-15-avril-2011-18045.htm.

[4] INVESTIR AU CAMEROUN. (2023). “Le Cameroun lance l’opérationnalisation de la Caisse des dépôts et consignations bloquée depuis 15 ans”. [En ligne], url: .

[5] CAMEROON TRIBUNE. (2023). “Caisse des dépôts et consignations : les premiers dirigeants nommés”. [En ligne], url: .

[6] ZABBIX LLC. *Zabbix Documentation 7.0*. Consulté le 3 septembre 2026. 2026. URL : https://www.zabbix.com/documentation/7.0/en/manual.

[7] David HARRINGTON, Randy PRESUHN et Bert WIJNEN. *An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks*. RFC 3411. Internet Engineering Task Force (IETF), décembre 2002. URL : https://www.rfc-editor.org/rfc/rfc3411.

[8] Jeffrey CASE et al. *Introduction and Applicability Statements for Internet-Standard Management Framework*. RFC 3410. Internet Engineering Task Force (IETF), décembre 2002. URL : https://www.rfc-editor.org/rfc/rfc3410.

[9] Randy PRESUHN. *Management Information Base (MIB) for the Simple Network Management Protocol (SNMP)*. RFC 3418. Internet Engineering Task Force (IETF), décembre 2002. URL : https://www.rfc-editor.org/rfc/rfc3418.

[10] Uri BLUMENTHAL et Bert WIJNEN. *User-based Security Model (USM) for version 3 of the Simple Network Management Protocol (SNMPv3)*. RFC 3414. Internet Engineering Task Force (IETF), décembre 2002. URL : https://www.rfc-editor.org/rfc/rfc3414.

[11] BROADCOM INC. *VMware Workstation Pro Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://docs.vmware.com/en/VMware-Workstation-Pro/.

[12] GNS3 TECHNOLOGIES INC. *GNS3 Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://docs.gns3.com/.

[13] VYOS NETWORKS. *VyOS Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://docs.vyos.io.

[14] NETGATE. *pfSense Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://docs.netgate.com/pfsense/en/latest/.

[15] OFFSEC SERVICES LIMITED. *Kali Linux Documentation*. Consulté le 5 septembre 2026. 2026. URL : https://www.kali.org/docs/.

[16] Gordon LYON. *Nmap Network Scanning — Official Reference Guide*. Consulté le 5 septembre 2026. 2026. URL : https://nmap.org/book/.

[17] THC (THE HACKER'S CHOICE). *THC-Hydra — Documentation*. Consulté le 5 septembre 2026. 2026. URL : https://github.com/vanhauser-thc/thc-hydra.

[18] Salvatore SANFILIPPO. *hping3 — Active Network Security Tool, Manual Page*. Consulté le 5 septembre 2026. 2026. URL : https://www.kali.org/tools/hping3/.

[19] NAGIOS ENTERPRISES. *Nagios Core Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://www.nagios.org/documentation/.

[20] CENTREON. *Centreon Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://docs.centreon.com/.

[21] PAESSLER AG. *PRTG Manual*. Consulté le 3 septembre 2026. 2026. URL : https://www.paessler.com/manuals/prtg.

[22] THE CACTI GROUP. *Cacti Documentation*. Consulté le 3 septembre 2026. 2026. URL : https://www.cacti.net/documentation.php.